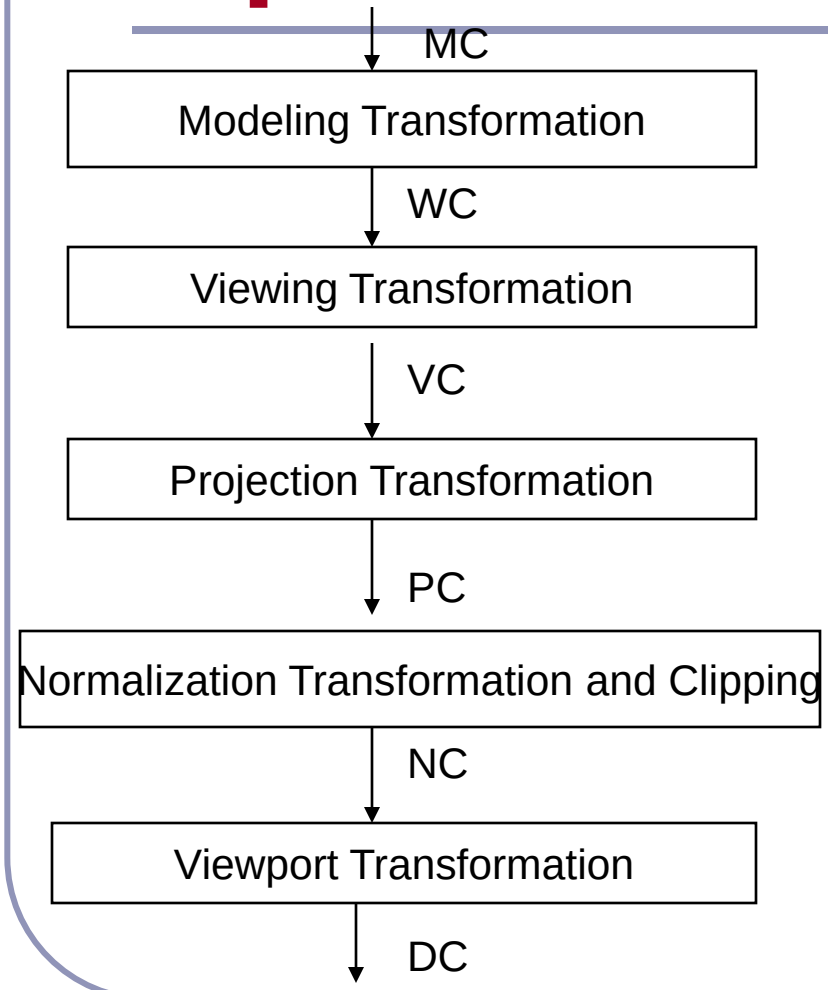
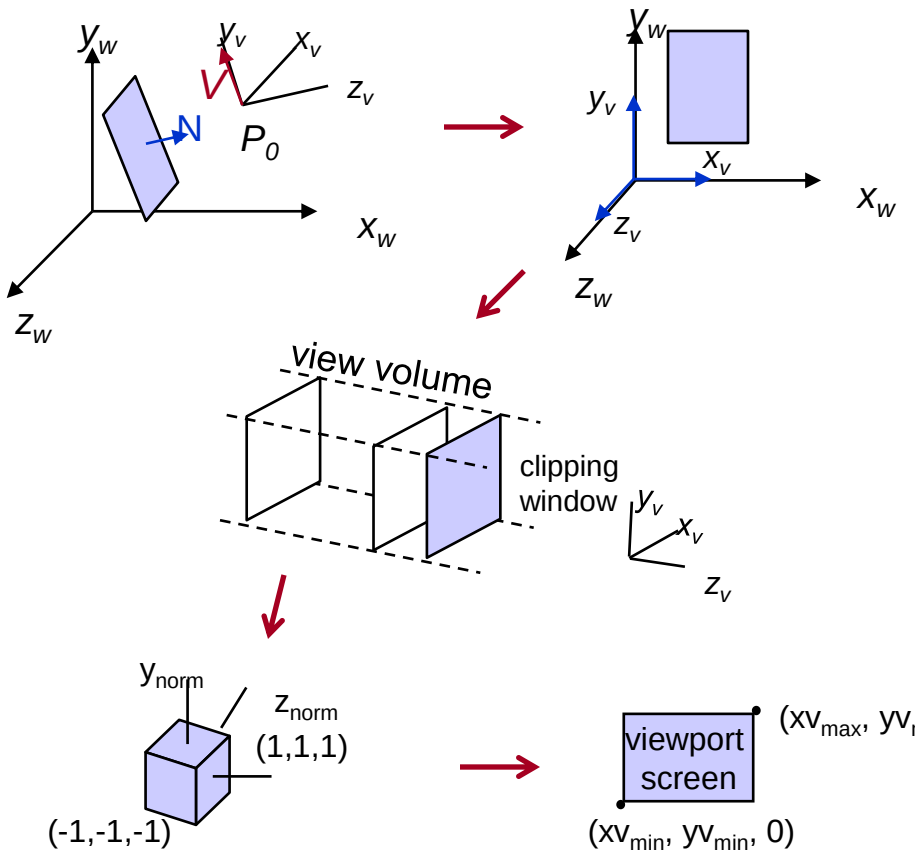
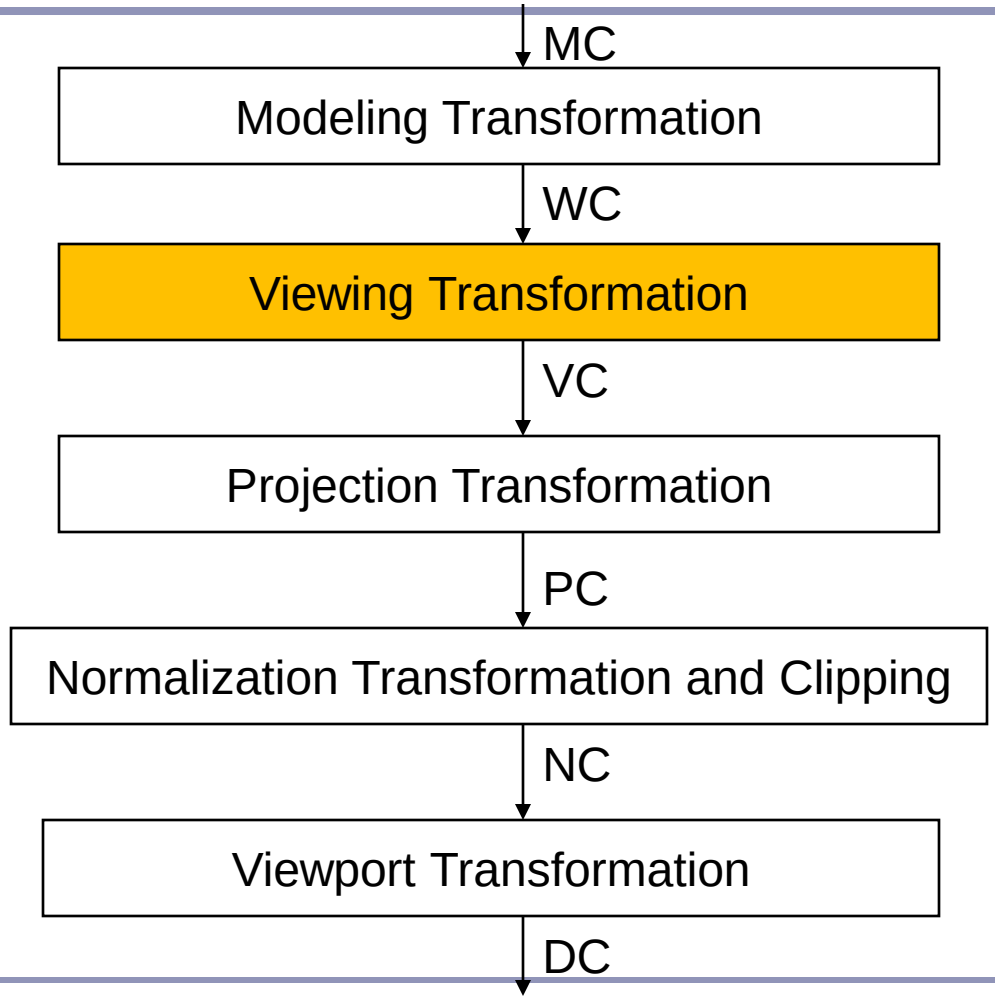
3D Viewing

3D Viewing Transformation Pipeline

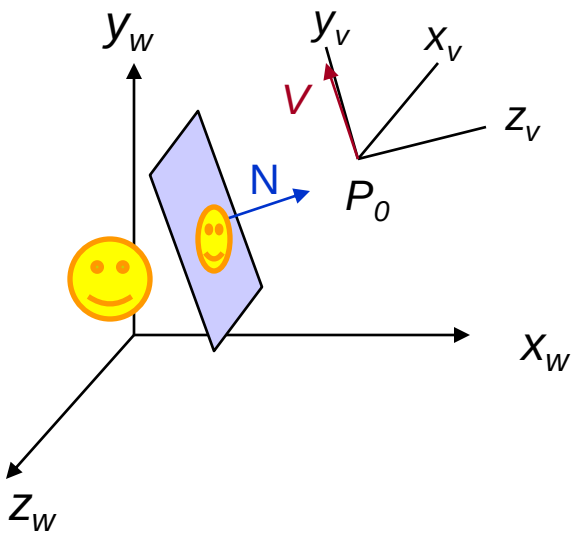


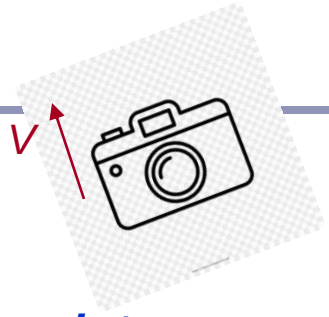


3D Viewing Transformation Pipeline



3D Viewing





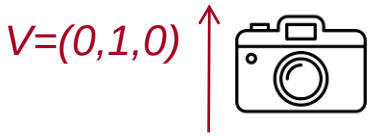
V ***view up vector***

$P_0 = (x_0, y_0, z_0)$ ***view point***

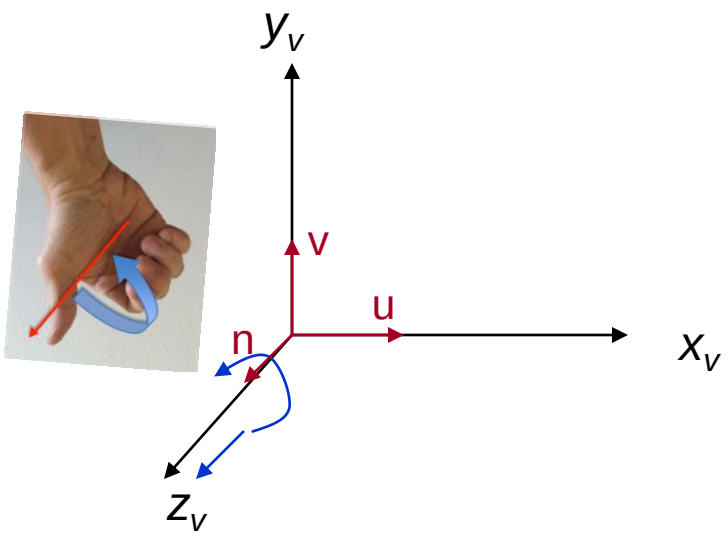
N ***viewplane normal***

Viewplane is at point z_{vp} in negative z_v direction

V is perpendicular to N

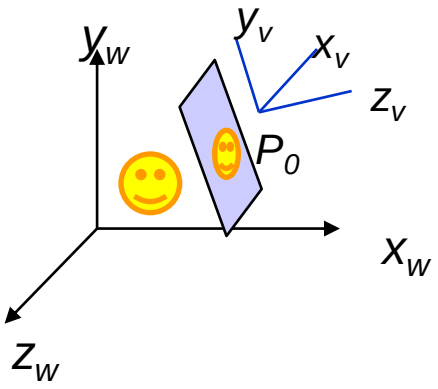


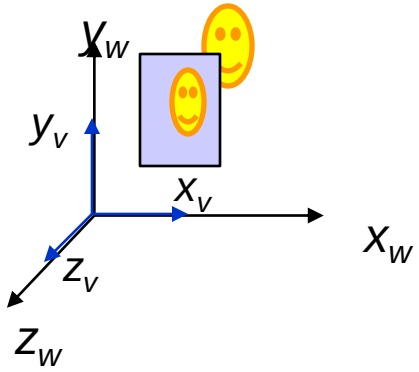
3D Viewing



x_v y_v z_v is a *right-handed* viewing coordinate system

3D Viewing





Transformation from World to Viewing Coordinates:

1. Translate viewing coordinate origin to the origin of world coordinate system

2. Apply rotations to align x_v y_v z_v axes with x_w y_w z_w axes respectively

$$M_{WC,VC} = R \cdot T$$

World to Viewing Coordinates Transformation

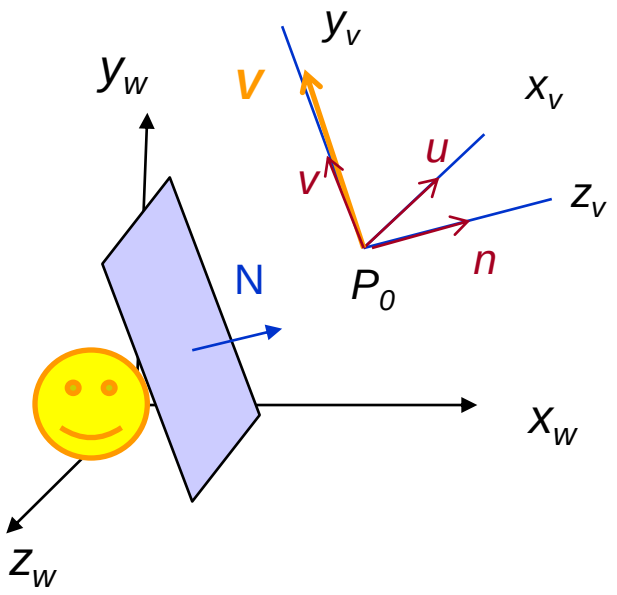
$M_{WC,VC} = R.T(-x_0, -y_0, -z_0)$

$$n = \frac{N}{|N|} = (n_x, n_y, n_z)$$

$$v = \frac{V}{|V|} = (v_x, v_y, v_z)$$

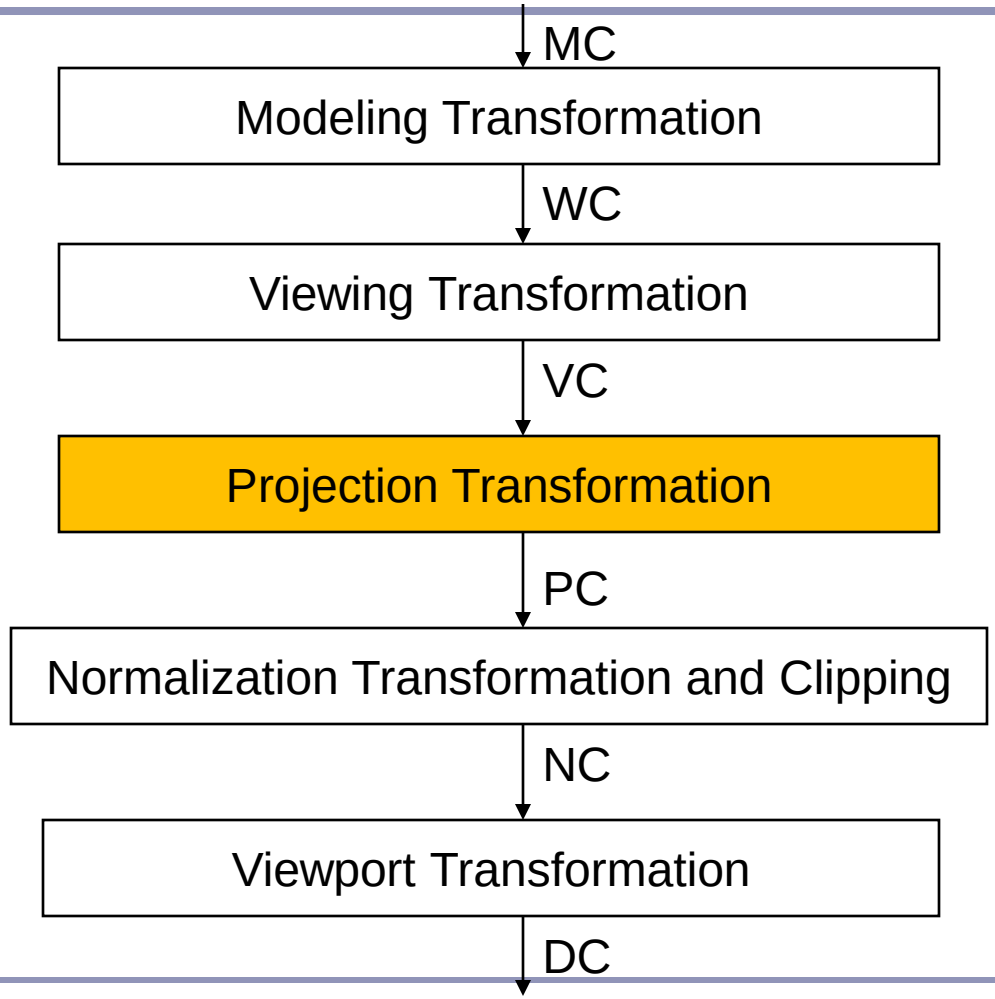
$$u = \frac{V}{|V|} \times \frac{N}{|N|} = (u_x, u_y, u_z)$$

$$R = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ n_x & n_y & n_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

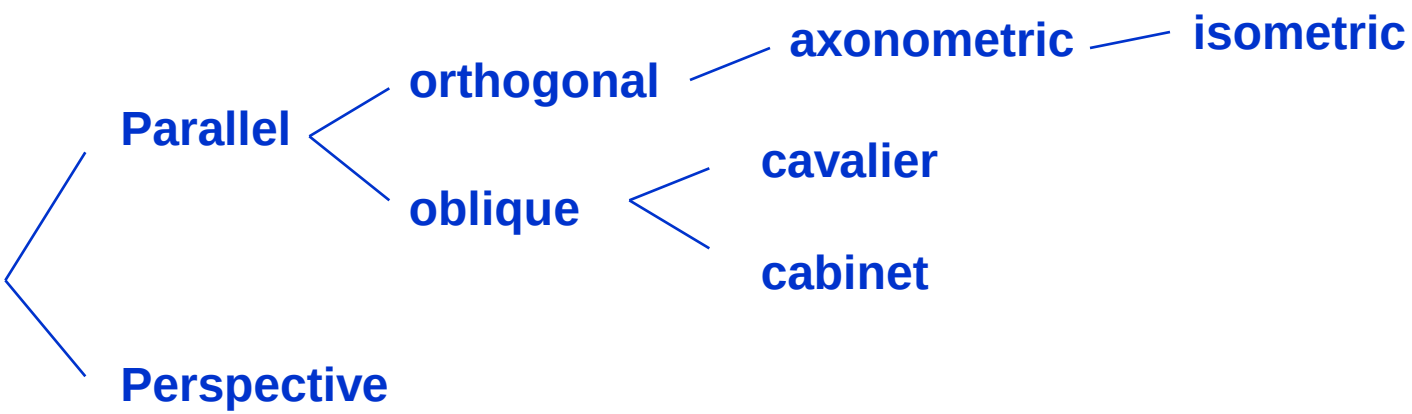


$P_0 = (x_0, y_0, z_0)$

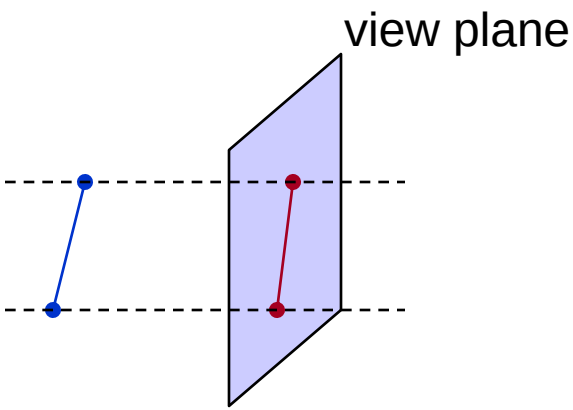
3D Viewing Transformation Pipeline



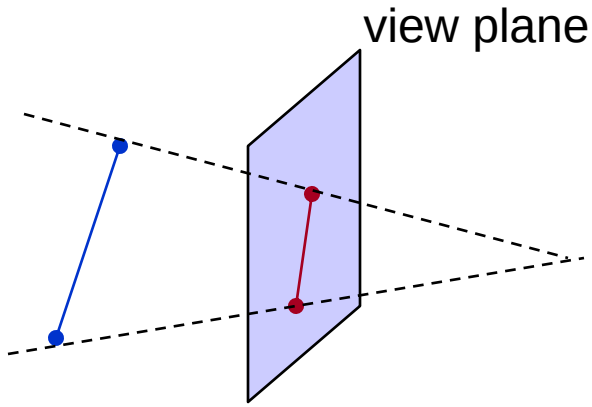
Projections



Projections

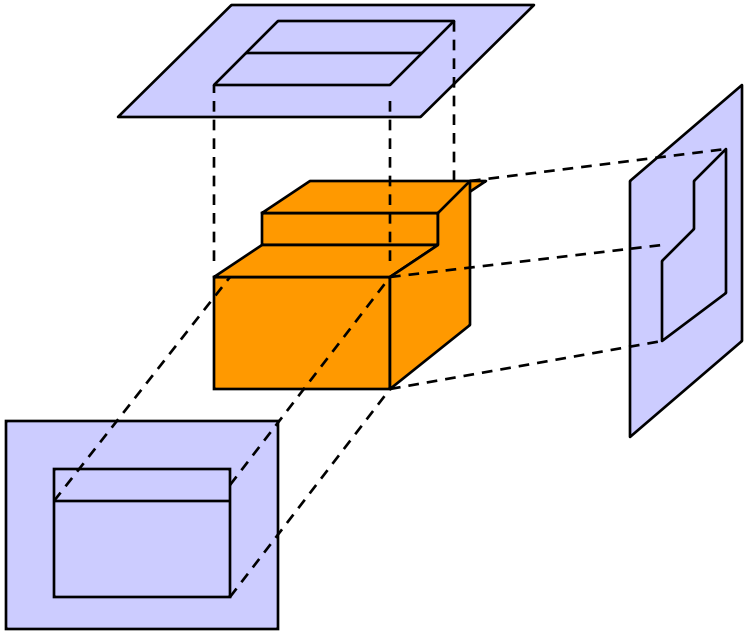


Parallel projection

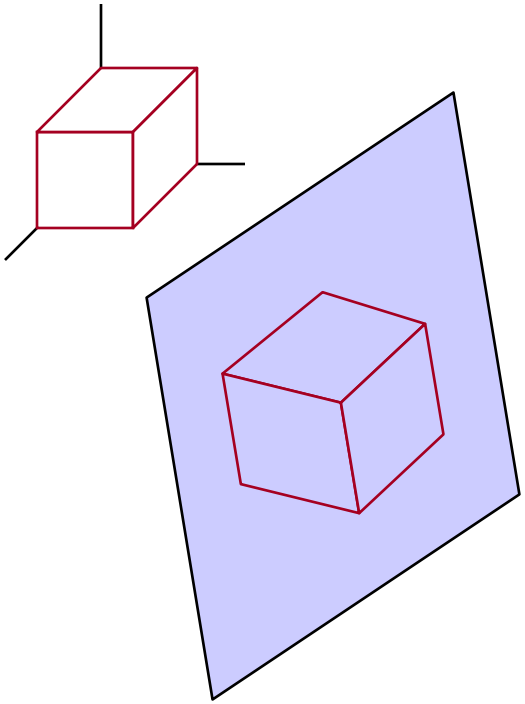


Perspective projection

Orthogonal Projection



Orthogonal Projection

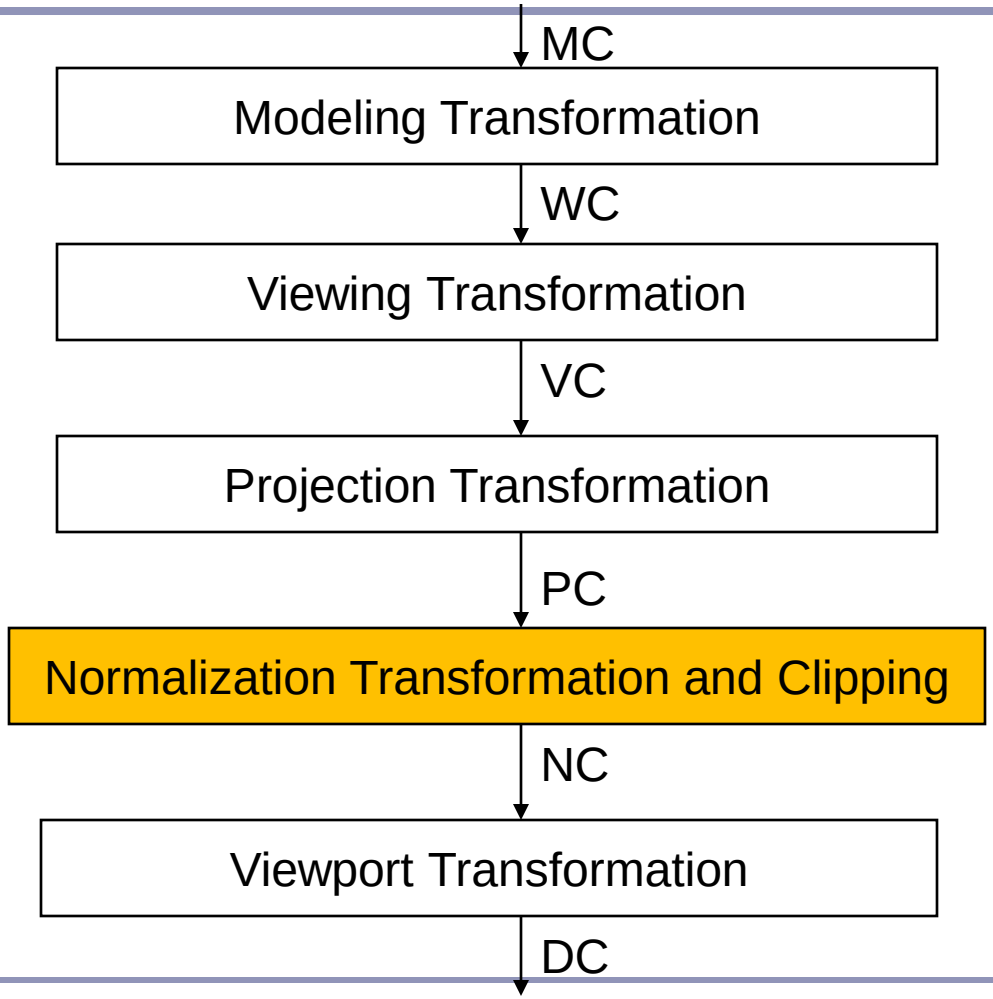


Isometric

Axonometric: ***displays more than one face of an object***

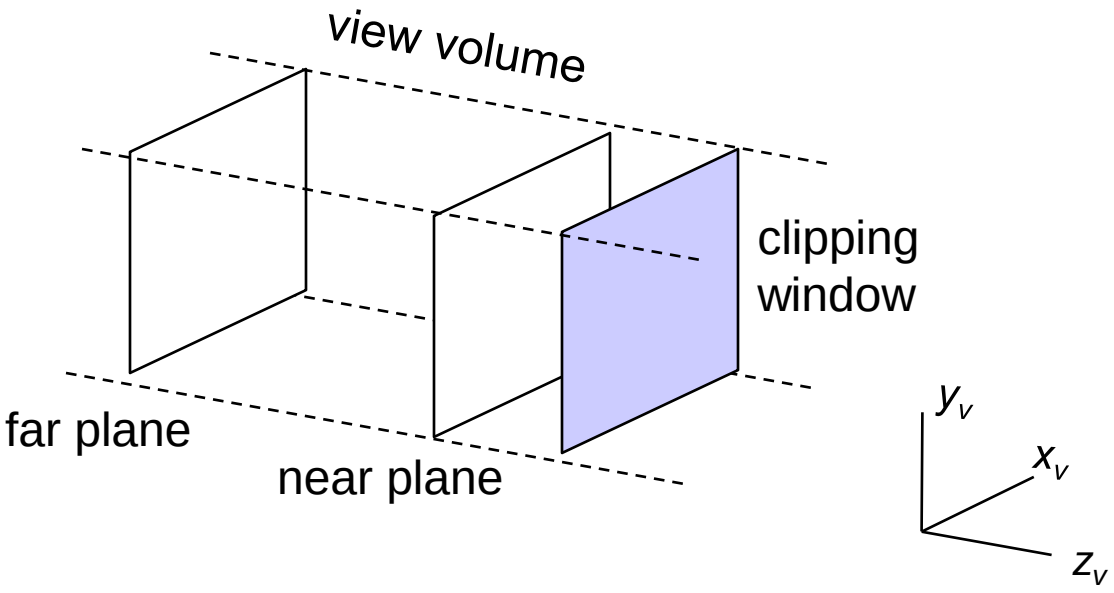
Isometric: ***projection plane intersects each coordinate axis at the same distance from the origin***

3D Viewing Transformation Pipeline



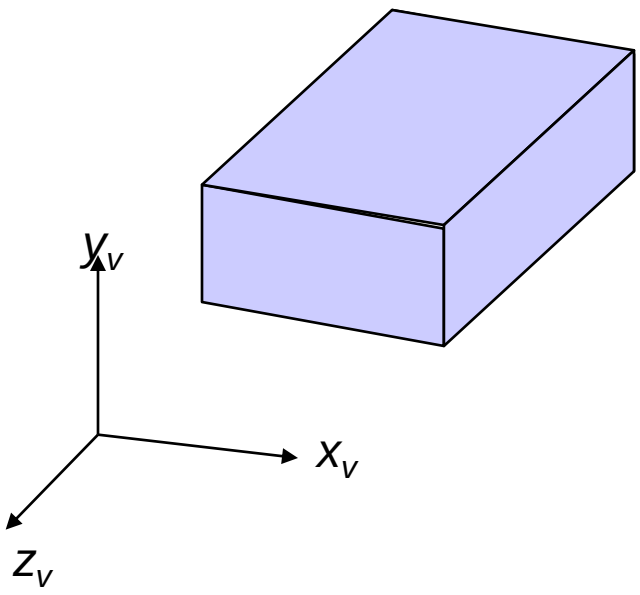
Orthogonal Projection

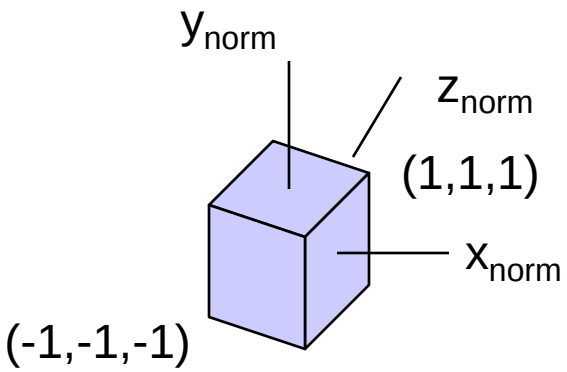
Clipping



Orthogonal Projection

Normalization

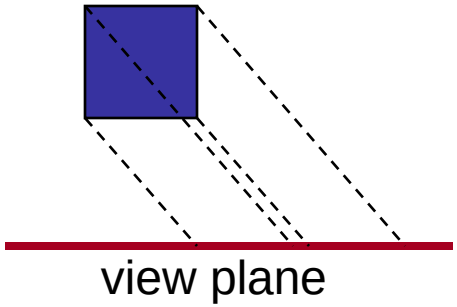


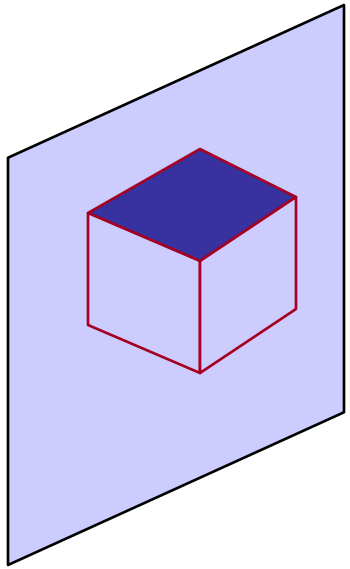


$$M_{ortho,norm} \cdot R \cdot T$$

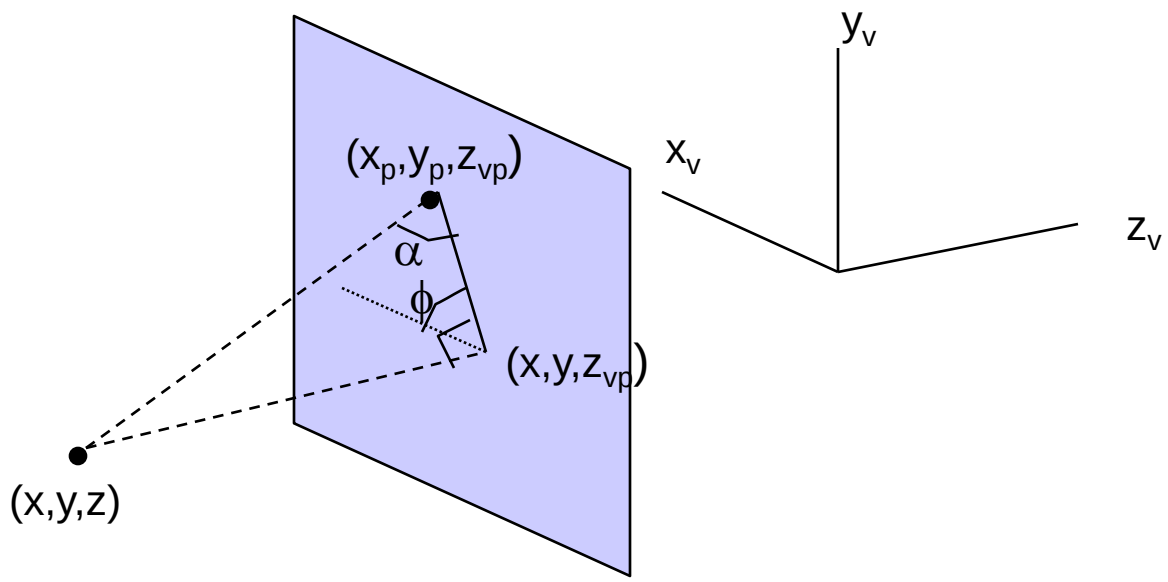
Oblique Projection

Oblique projection: ***projection path is not perpendicular to the view plane***





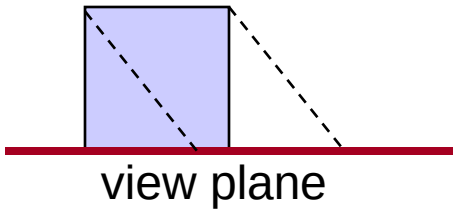
Oblique Projection

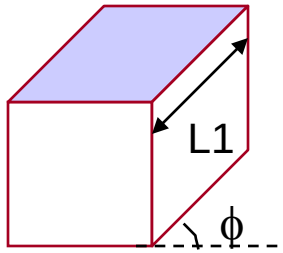


Oblique Projection

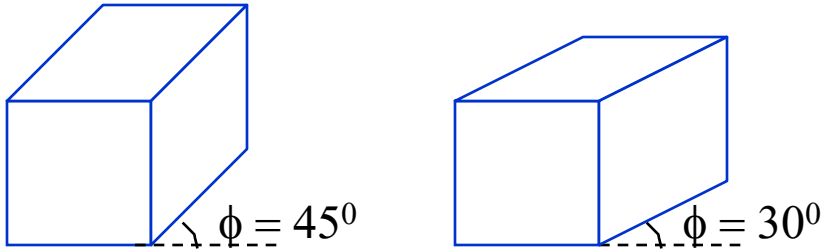
$$x_p = x + L_1 (z_{vp} - z) \cos \phi$$
$$y_p = y + L_1 (z_{vp} - z) \sin \phi$$

shearing transformation

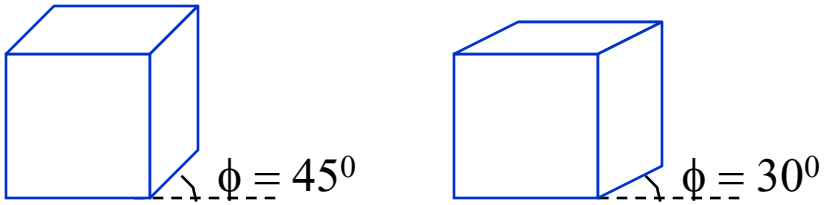




Oblique Projection

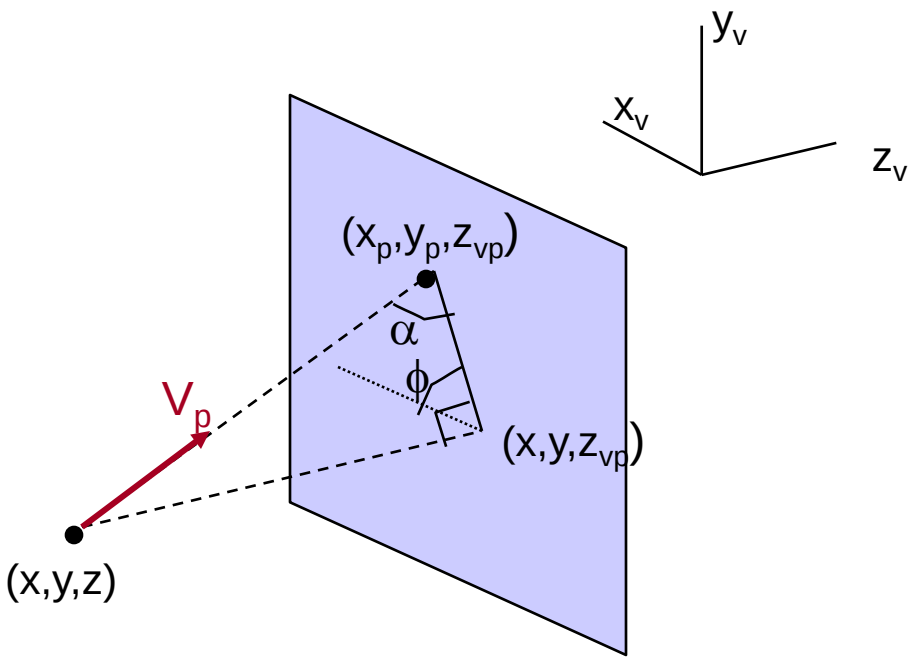


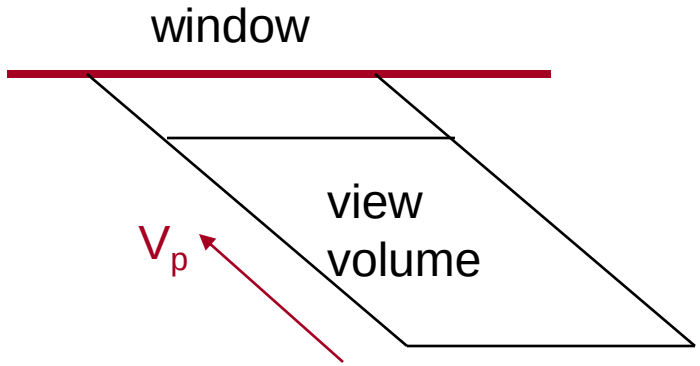
Cavalier Projection



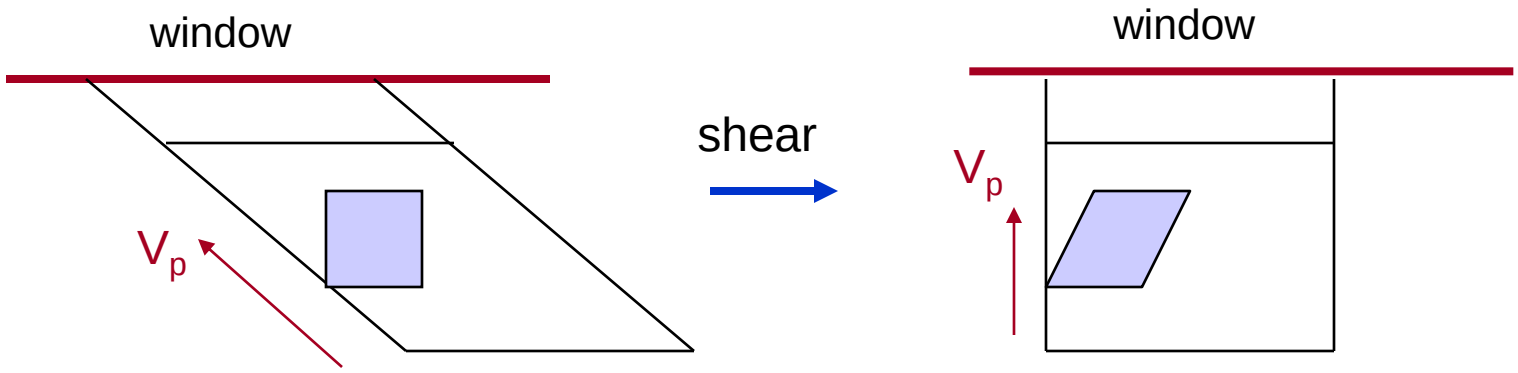
Cabinet Projection

Oblique Projection



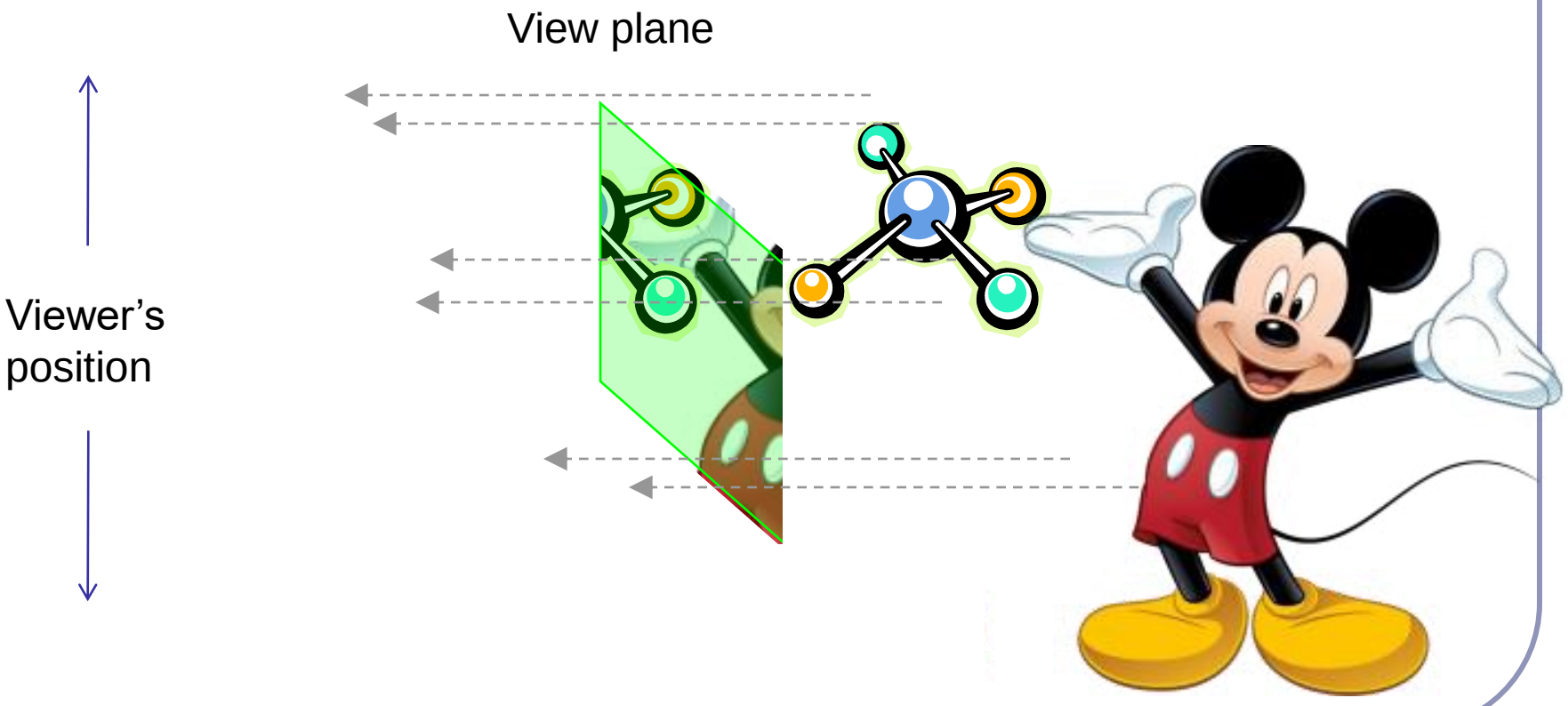


Oblique Projection

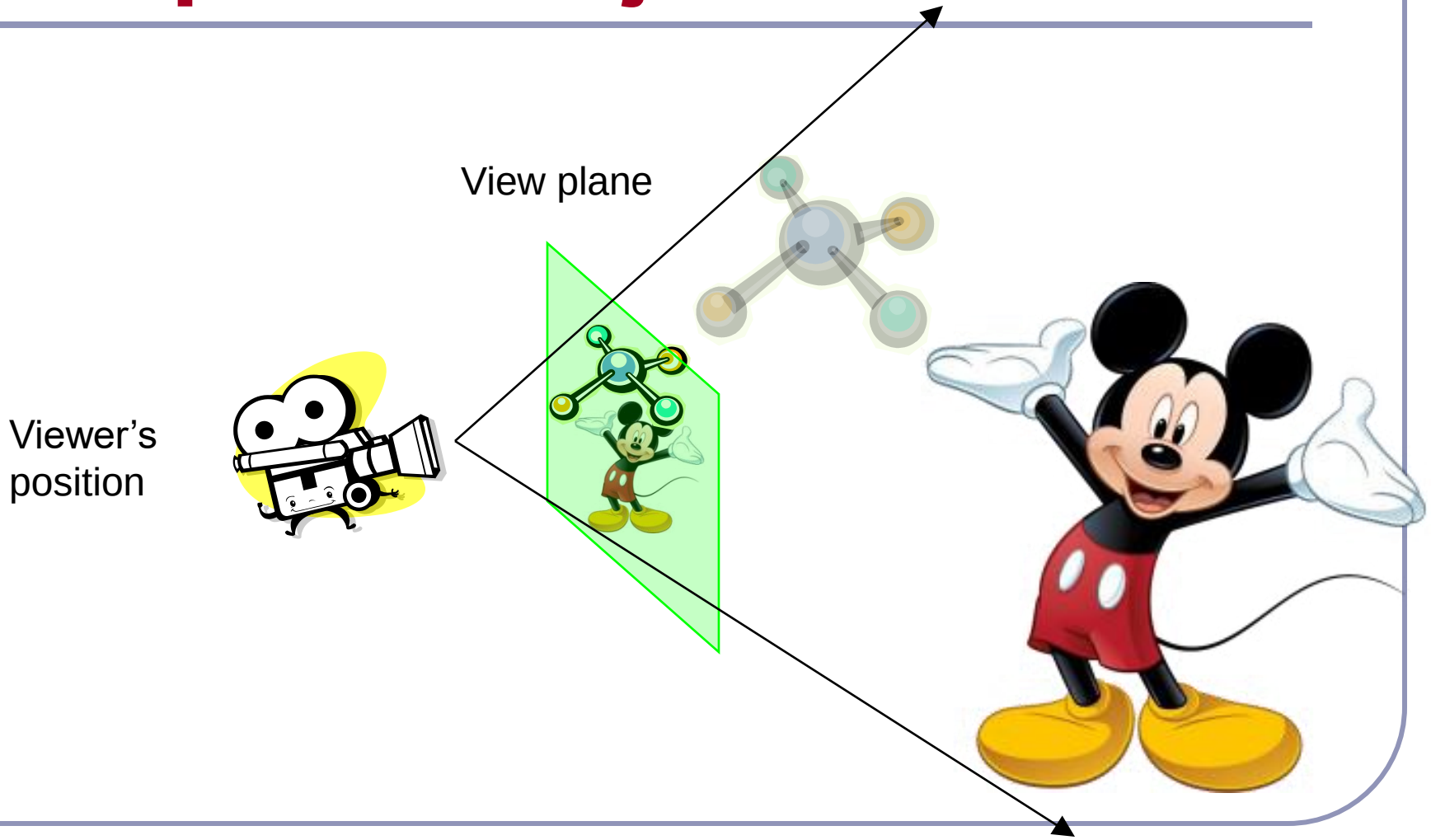


$$M_{WC,VC} \cdot \underbrace{M_{orth,norm} \cdot M_{obl}}_{M_{obl,norm}}$$

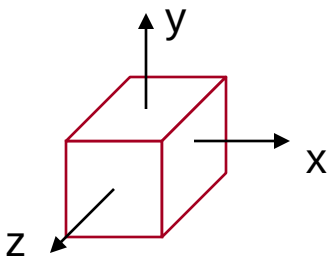
Parallel Projection

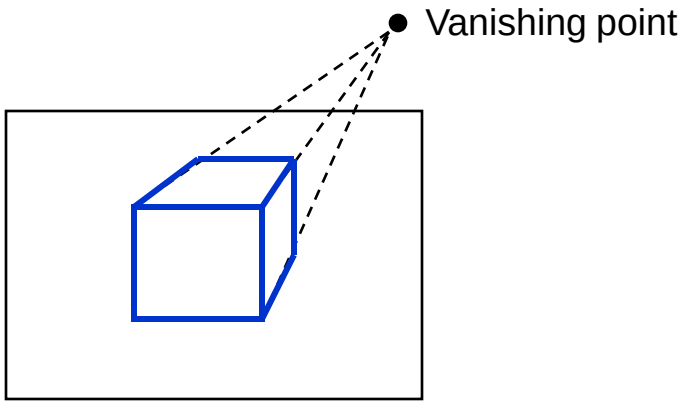


Perspective Projection

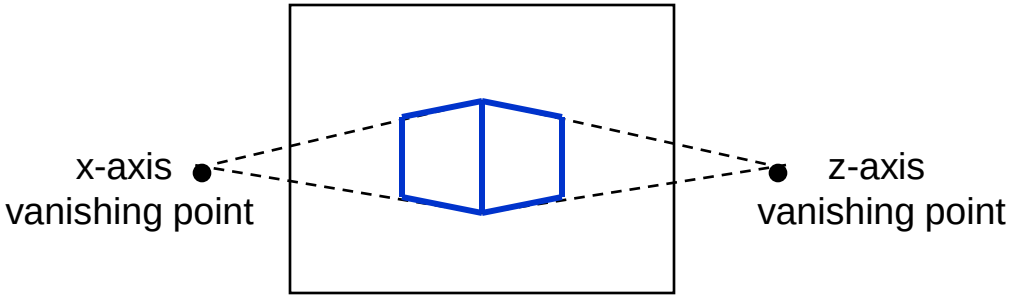


Perspective Projection



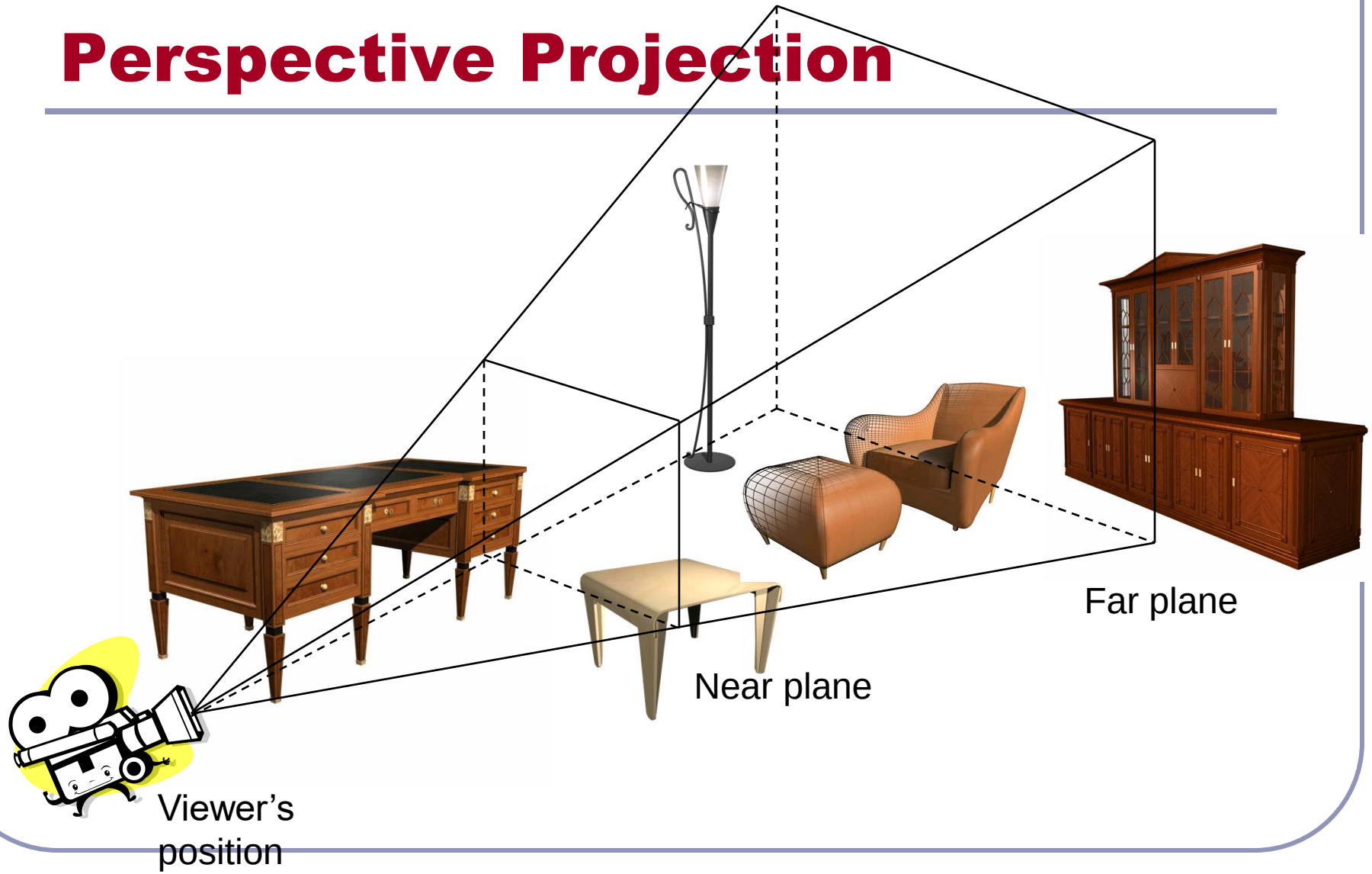


One-point perspective projection

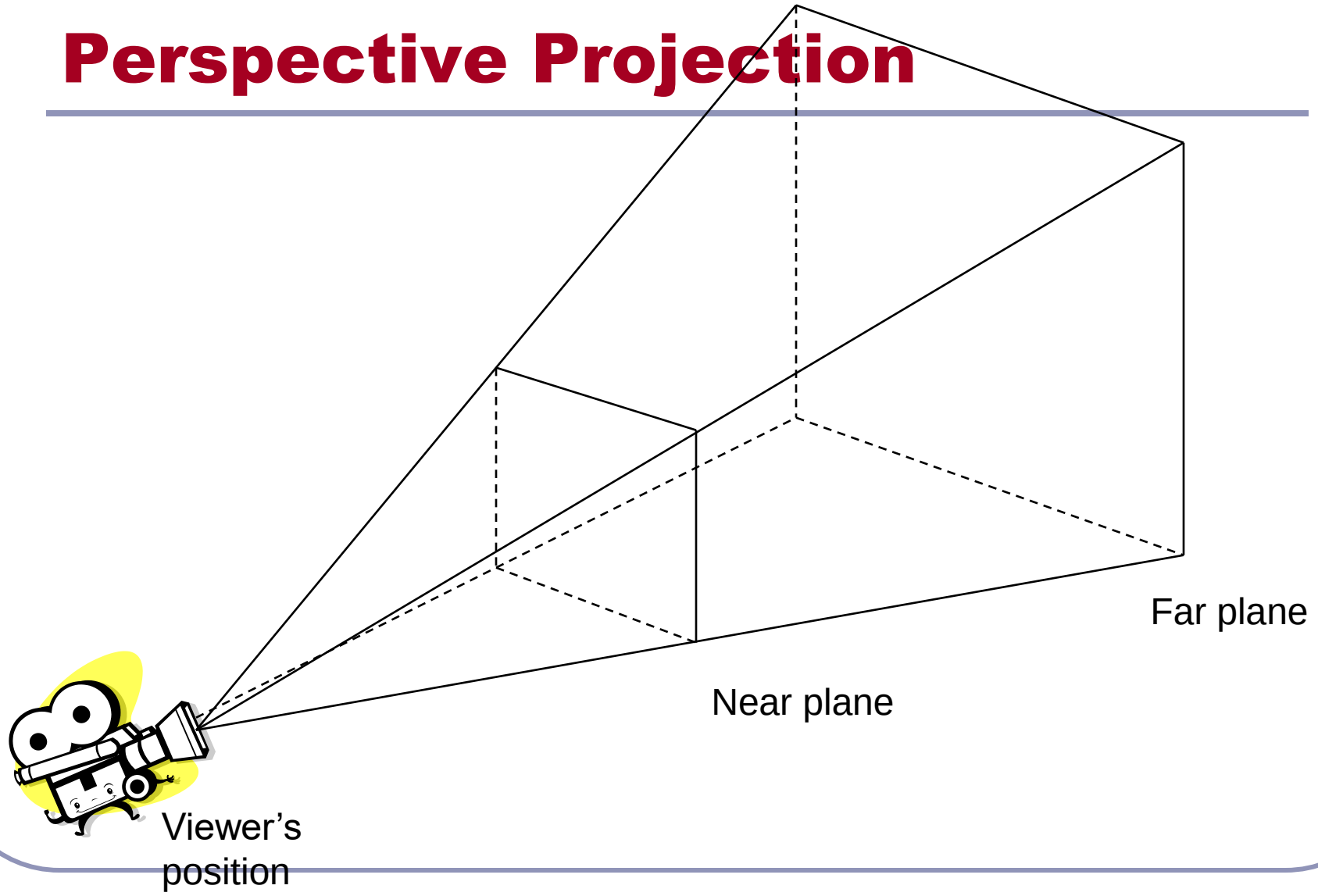


Two-point perspective projection

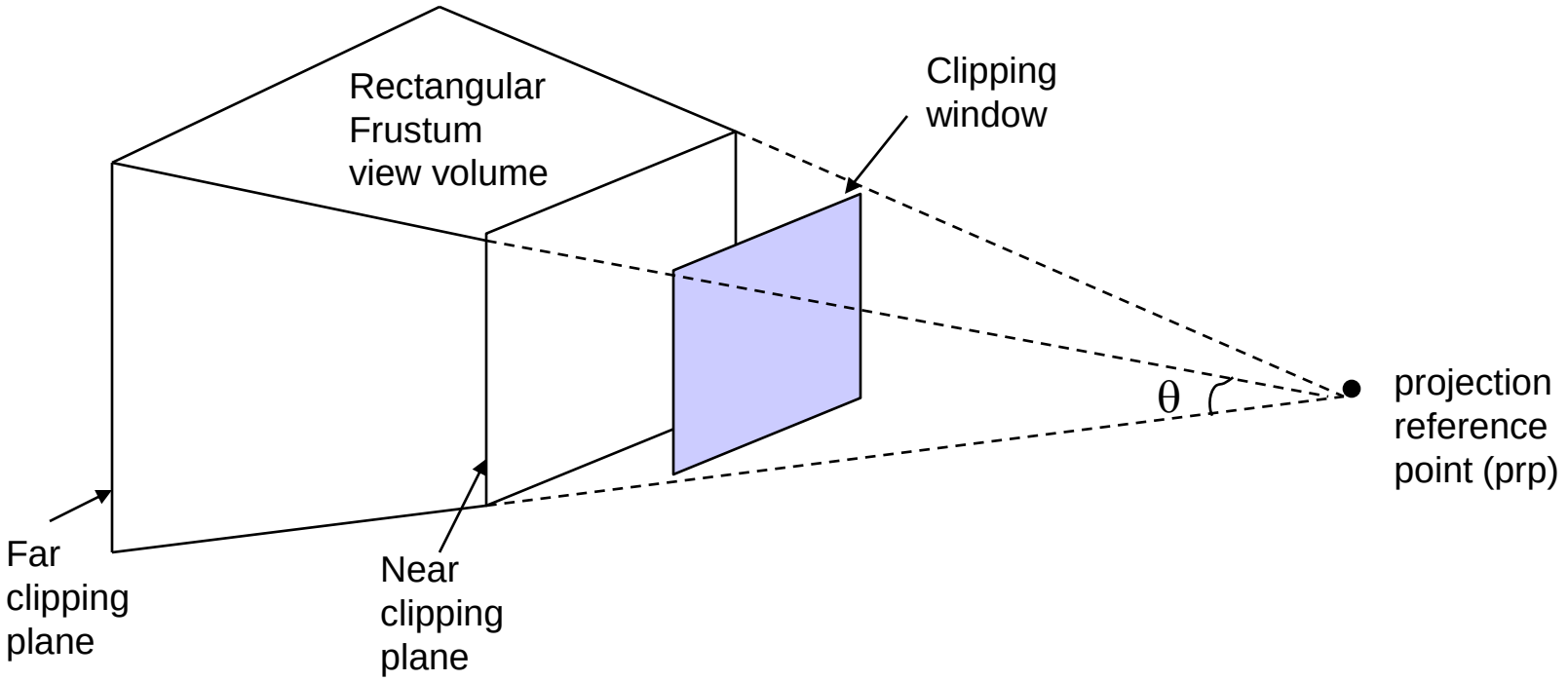
Perspective Projection



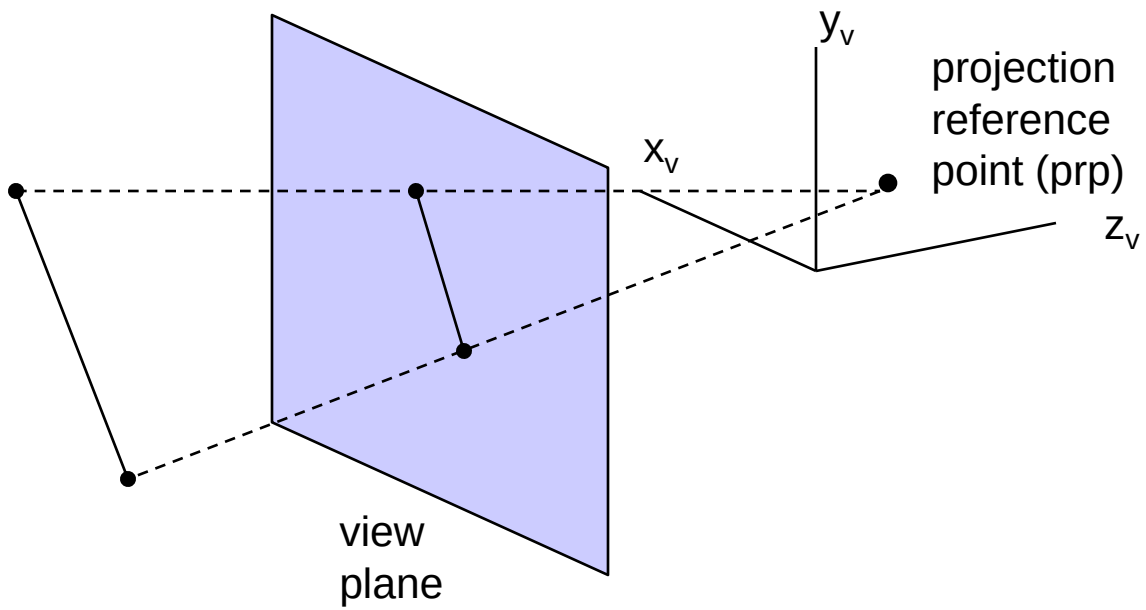
Perspective Projection



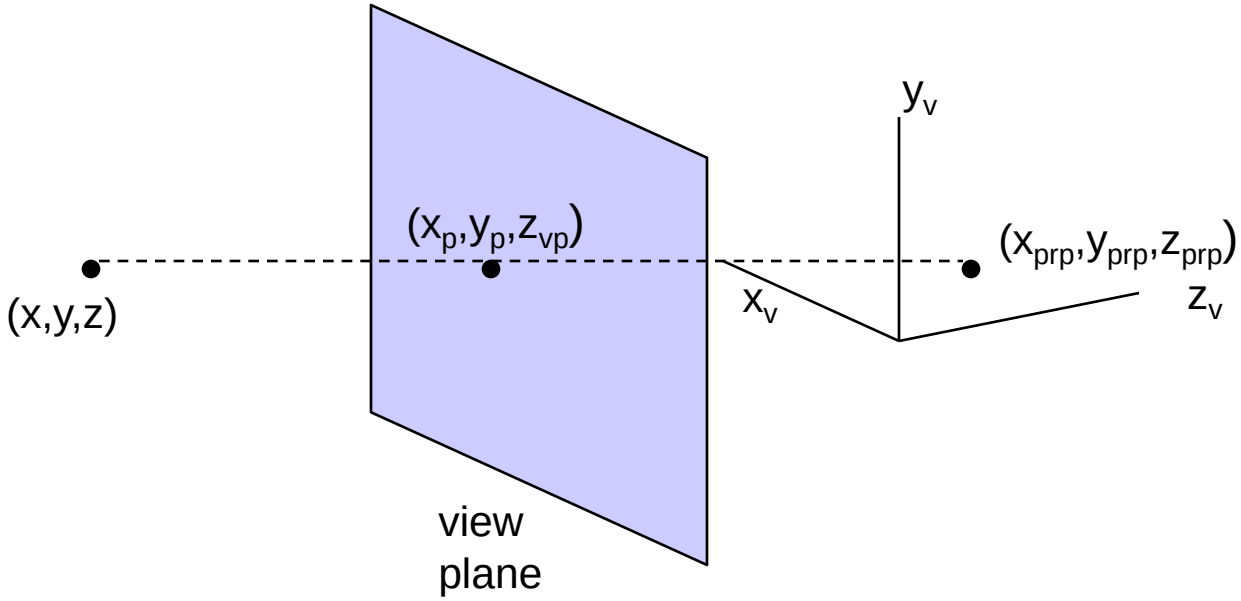
Perspective Projection



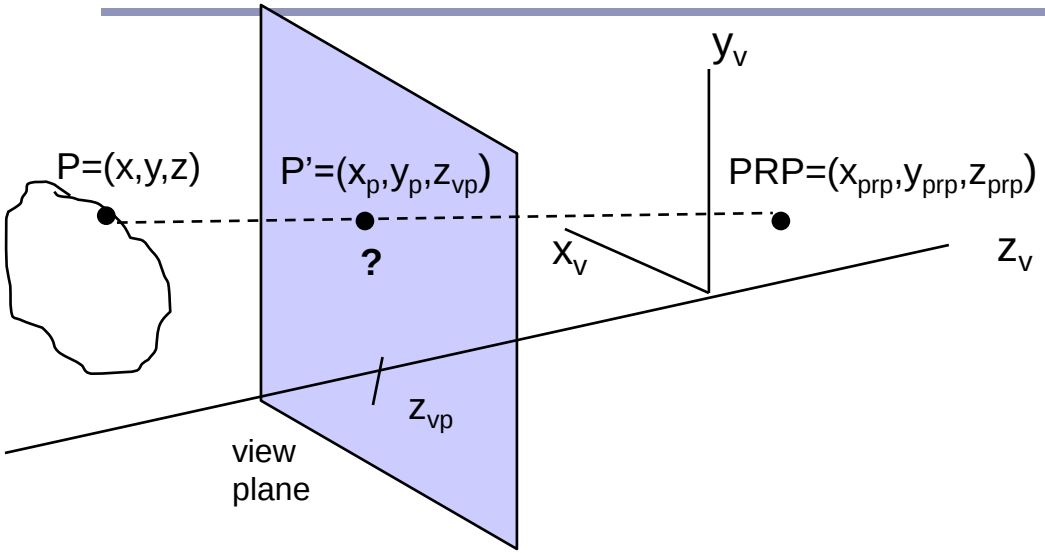
Perspective Projection



Perspective Projection



Perspective Projection



$$x_p = x - u(x - x_{prp})$$

$$y_p = y - u(y - y_{prp})$$

$$z_p = z - u(z - z_{prp}) = z_{vp}$$

$$u = \frac{(z_{vp} - z)}{(z_{prp} - z)}$$

$$x_p = x\frac{(z_{prp} - z_{vp})}{(z_{prp} - z)} + x_{prp}\frac{(z_{vp} - z)}{(z_{prp} - z)}$$

$$y_p = y\frac{(z_{prp} - z_{vp})}{(z_{prp} - z)} + y_{prp}\frac{(z_{vp} - z)}{(z_{prp} - z)}$$

$$z_p = z_{vp}$$

Perspective Projection

$$h = z_{prp} - z$$

$$x_p = x\frac{(z_{prp} - z_{vp})}{(z_{prp} - z)} + x_{prp}\frac{(z_{vp} - z)}{(z_{prp} - z)}$$

$$x_p = \frac{x(z_{prp} - z_{vp}) + x_{prp}(z_{vp} - z)}{h} = \frac{x_h}{h}$$

$$x_h = x(z_{prp} - z_{vp}) + x_{prp}(z_{vp} - z)$$
$$= x(z_{prp} - z_{vp}) - z.x_{prp} + z_{vp}.x_{prp}$$

$$y_p = y\frac{(z_{prp} - z_{vp})}{(z_{prp} - z)} + y_{prp}\frac{(z_{vp} - z)}{(z_{prp} - z)}$$

$$y_p = \frac{y(z_{prp} - z_{vp}) + y_{prp}(z_{vp} - z)}{h} = \frac{y_h}{h}$$

$$y_h = y(z_{prp} - z_{vp}) + y_{prp}(z_{vp} - z)$$
$$= y(z_{prp} - z_{vp}) - z.y_{prp} + z_{vp}.y_{prp}$$

$$z_p = \frac{z_h}{h} = \frac{z_{vp}}{h}$$

Perspective Projection

$P = (x, y, z, 1) \rightarrow P_h = (x_h, y_h, z_h, h)$

$$x_h = x(z_{prp} - z_{vp}) - z.x_{prp} + x_{prp}z_{vp}$$

$$y_h = y(z_{prp} - z_{vp}) - z.y_{prp} + y_{prp}z_{vp}$$

M_{pers} P P_h

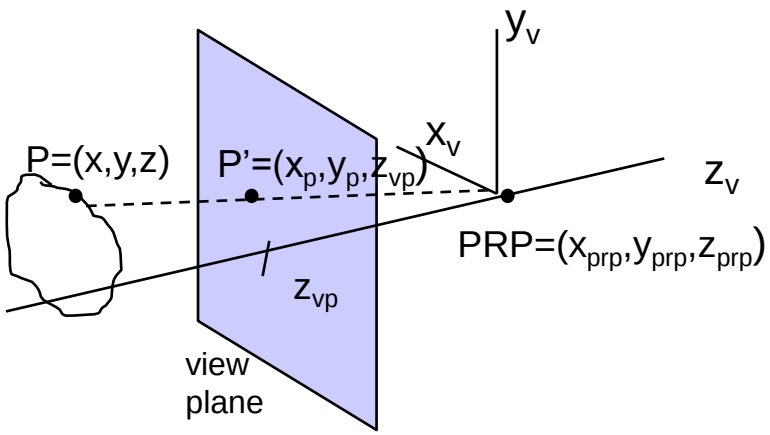
$$\begin{bmatrix} z_{prp} - z_{vp} & 0 & -x_{prp} & x_{prp}z_{vp} \\ 0 & z_{prp} - z_{vp} & -y_{prp} & y_{prp}z_{vp} \\ 0 & 0 & S_z & t_z \\ 0 & 0 & -1 & z_{prp} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \rightarrow \begin{bmatrix} x_h \\ y_h \\ z_h \\ h \end{bmatrix}$$

$P_h = M_{pers} \cdot P$

Perspective Projection

PRP is at (0,0,0)

$$M_{pers} = \begin{bmatrix} -z_{vp} & 0 & 0 & 0 \\ 0 & -z_{vp} & 0 & 0 \\ 0 & 0 & S_z & t_z \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

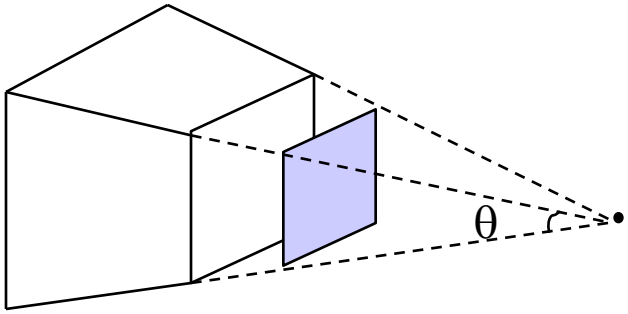


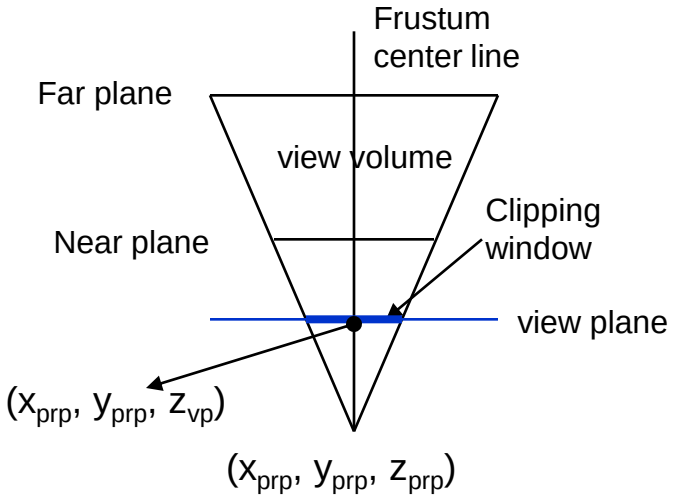
$$\begin{bmatrix} x_h \\ y_h \\ z_h \\ h \end{bmatrix} = M_{pers} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$P_h = M_{pers} \cdot P$

Perspective Projection

Symmetric Frustum

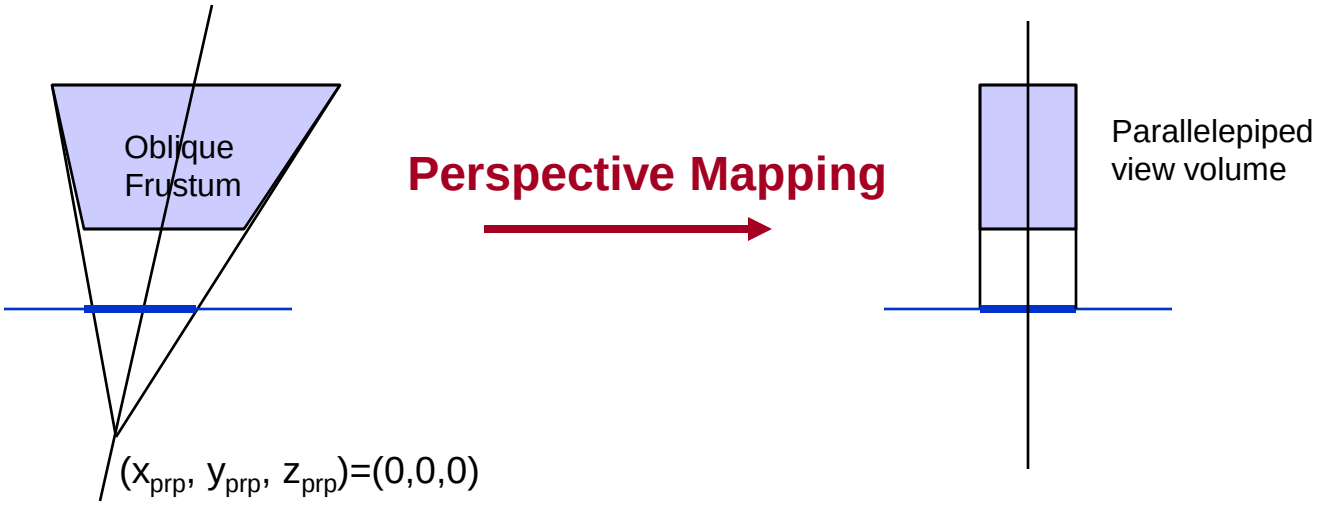




Perspective Projection

Then apply normalization transformation

Perspective Projection



1. Transform to a symmetric frustum (z-axis shear)

2. Normalize viewvolume

Perspective Projection

PRP is at (0,0,0)

$$M_{zshear} = \begin{bmatrix} 1 & 0 & shear_x & 0 \\ 0 & 1 & shear_y & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$shear_x = \frac{x_{w\,min} + x_{w\,max}}{2}$$

$$shear_y = \frac{y_{w\,min} + y_{w\,max}}{2}$$

Perspective Projection

PRP is at (0,0,0)

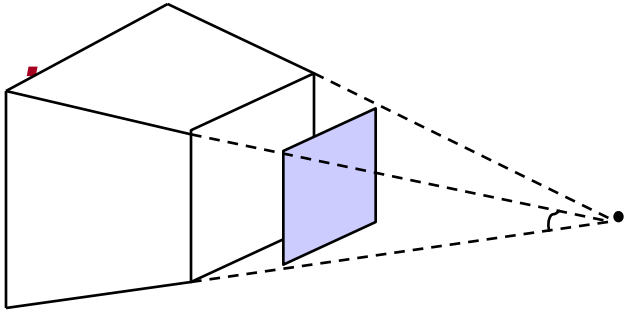
$z_{vp} = z_{near}$ (assuming that the viewplane is at the position of the near plane)

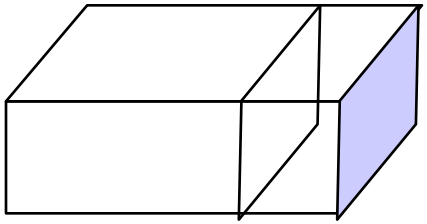
$M_{oblpers} = M_{pers} \cdot M_{zshear}$

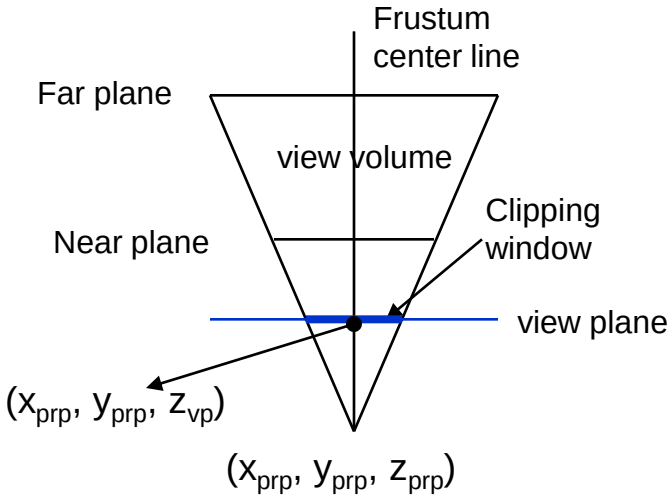
$$M_{oblpers} = \begin{bmatrix} -z_{near} & 0 & \frac{x_{wmin} + x_{wmax}}{2} & 0 \\ 0 & -z_{near} & \frac{y_{wmin} + y_{wmax}}{2} & 0 \\ 0 & 0 & S_z & t_z \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

$P_h = M_{oblpers} \cdot P$

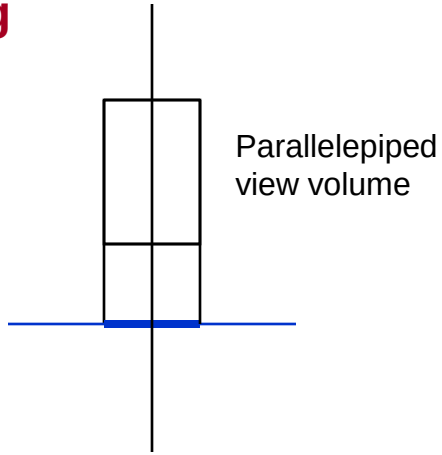
Perspective Projection





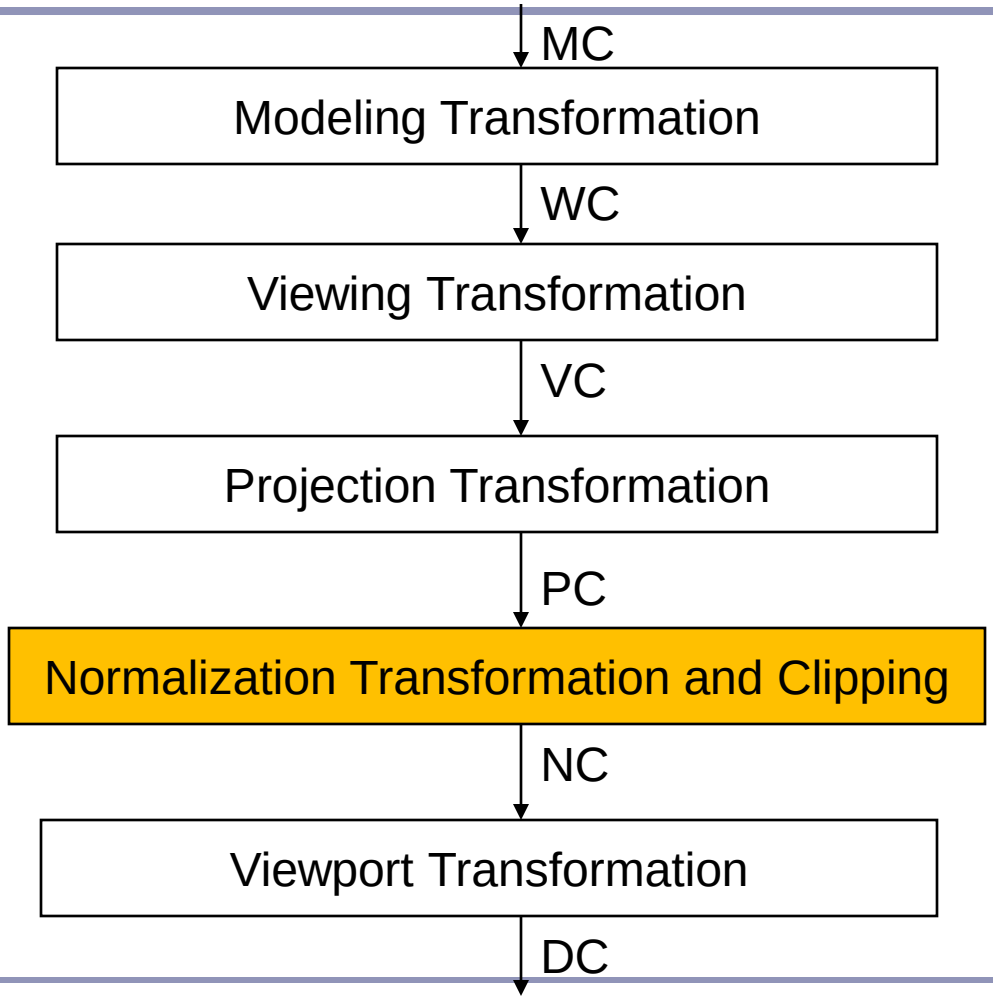


Perspective Mapping



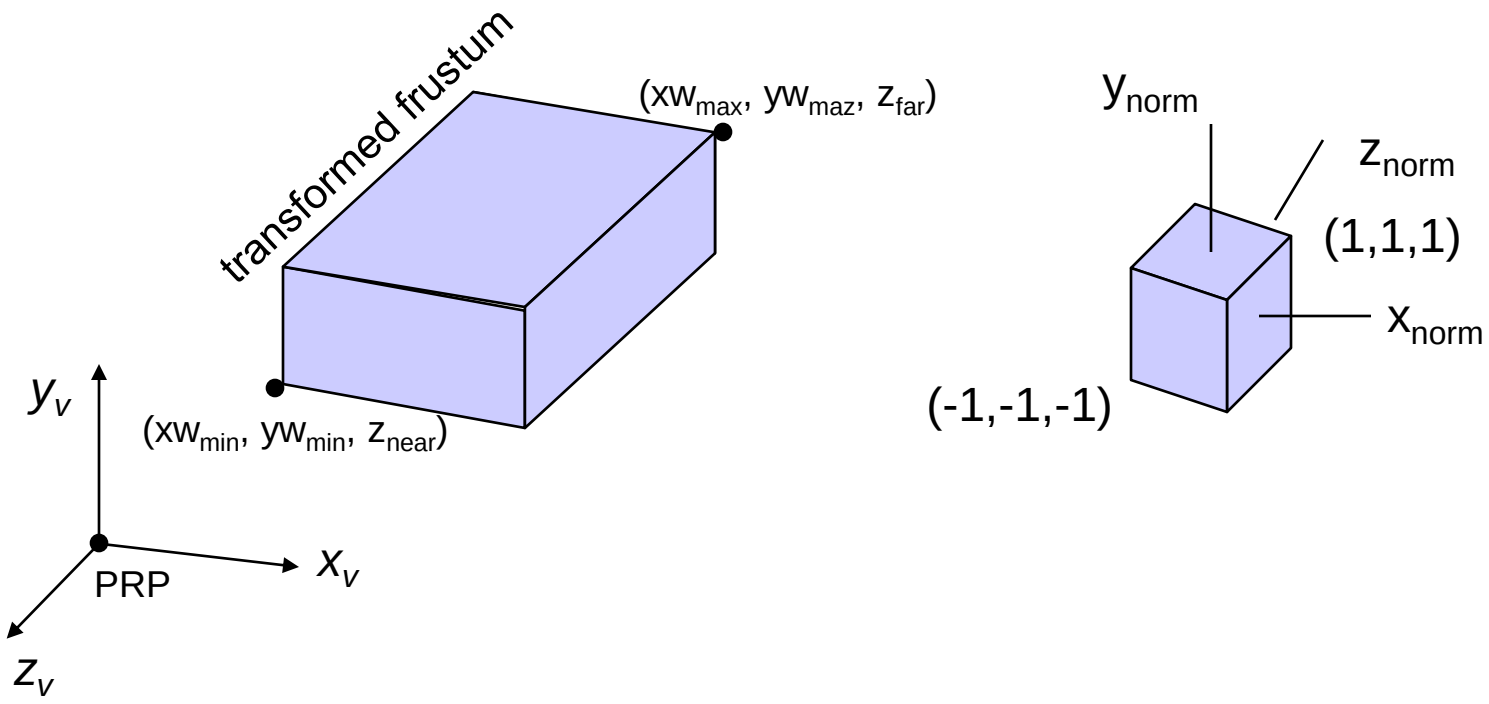
Then apply normalization transformation

3D Viewing Transformation Pipeline



Perspective Projection

Normalization



Perspective Projection

$M_{normpers} = M_{xyscale} \cdot M_{oblpers}$

$$M_{xyscale} = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$M_{normpers} = \begin{bmatrix} -z_{near} S_x & 0 & \frac{x_{w\,min} + x_{w\,max}}{2} S_x & 0 \\ 0 & -z_{near} S_y & \frac{y_{w\,min} + y_{w\,max}}{2} S_y & 0 \\ 0 & 0 & S_z & t_z \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Perspective Projection

$$\begin{bmatrix} x_h \\ y_h \\ z_h \\ h \end{bmatrix} = \begin{bmatrix} -z_{near}S_x & 0 & \frac{x_{wmin}+x_{wmax}}{2}S_x & 0 \\ 0 & -z_{near}S_y & \frac{y_{wmin}+y_{wmax}}{2}S_y & 0 \\ 0 & 0 & S_z & t_z \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$$x_h = -z_{near}S_x x + S_x \frac{xw_{min}+xw_{max}}{2} z$$

$$y_h = -z_{near}S_y y + S_y \frac{yw_{min}+yw_{max}}{2} z$$

$$z_h = S_z z + t_z$$

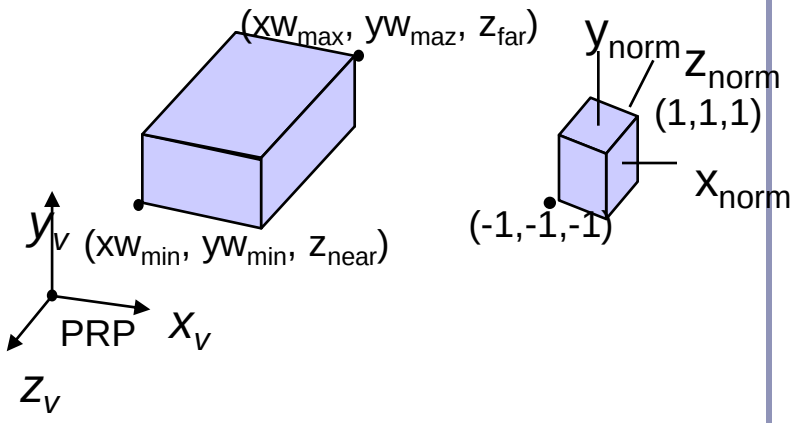
$$h = -z$$

$$x_p = \frac{x_h}{h} = \frac{-z_{near}S_x x + S_x \frac{xw_{min}+xw_{max}}{2} z}{-z}$$

$$y_p = \frac{y_h}{h} = \frac{-z_{near}S_y y + S_y \frac{yw_{min}+yw_{max}}{2} z}{-z}$$

Perspective Projection

$$x_p = \frac{-z_{near} S_x x + S_x \frac{xw_{min} + xw_{max}}{2} z}{-z}$$



(x_{wmin}, y_{wmin}, z_{wmin}) → (-1, -1, -1)

at $z=z_{near}$ $\quad x_p = -1$

$$-1 = \frac{-z_{near} S_x \mathbf{xw_{min}} + S_x \frac{xw_{min} + xw_{max}}{2} \mathbf{z_{near}}}{-\mathbf{z_{near}}}$$

$$S_x = \frac{2}{xw_{max} - xw_{min}}$$

similarly for y : $\quad S_y = \frac{2}{yw_{max} - yw_{min}}$

Perspective Projection

$$z_h = S_z z + t_z$$

$$z_p = \frac{z_h}{h} = \frac{S_z z + t_z}{-z}$$

at $z=z_{near}$ $\quad z_p = -1$

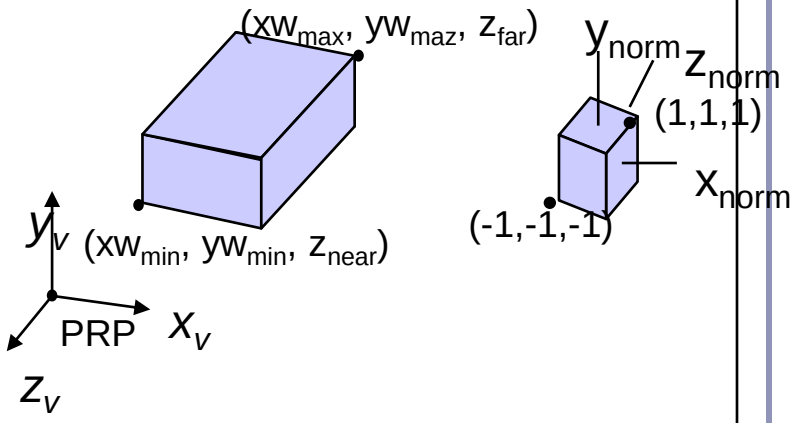
$$-1 = \frac{S_z \mathbf{z_{near}} + t_z}{-\mathbf{z_{near}}}$$

at $z=z_{far}$ $\quad z_p = 1$

$$1 = \frac{S_z \mathbf{z_{far}} + t_z}{-\mathbf{z_{far}}}$$

$$S_z = \frac{z_{near} + z_{far}}{z_{near} - z_{far}}$$

$$t_z = \frac{2 z_{near} z_{far}}{z_{near} - z_{far}}$$



Perspective Projection

$$M_{normpers} = \begin{bmatrix} -z_{near}S_x & 0 & \frac{xw_{min}+xw_{max}}{2}S_x & 0 \\ 0 & -z_{near}S_y & \frac{yw_{min}+yw_{max}}{2}S_y & 0 \\ 0 & 0 & S_z & t_z \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

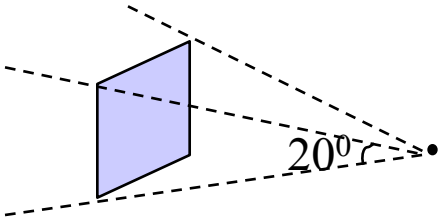
$$\begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix} = M_{normpers} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

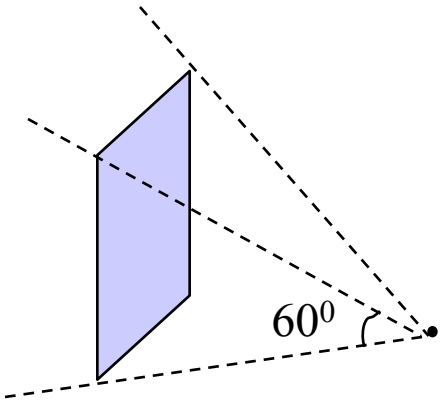
$$S_x = \frac{2}{xw_{max} - xw_{min}} \qquad S_z = \frac{z_{near} + z_{far}}{z_{near} - z_{far}}$$

$$S_y = \frac{2}{yw_{max} - yw_{min}} \qquad t_z = \frac{2z_{near}z_{far}}{z_{near} - z_{far}}$$

Perspective Projection

.





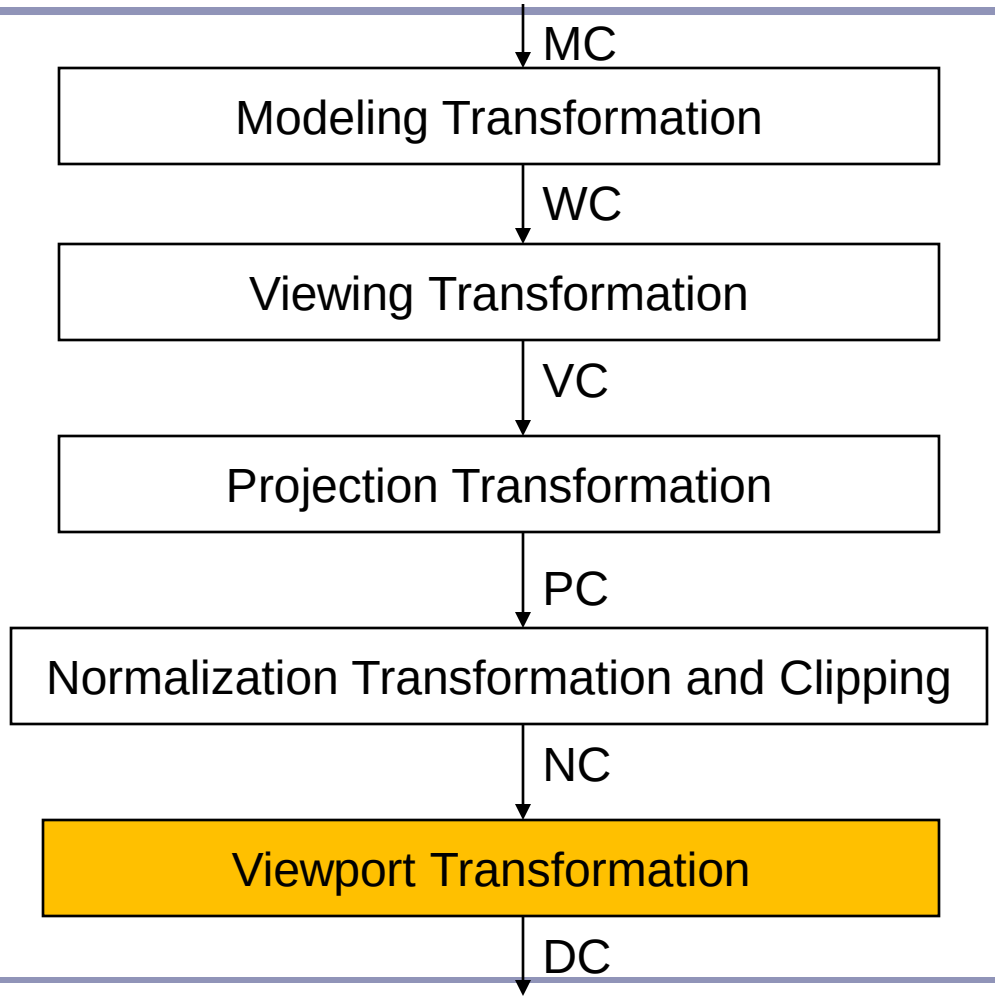
Perspective Projection

For symmetric frustum with field-of-view angle θ:

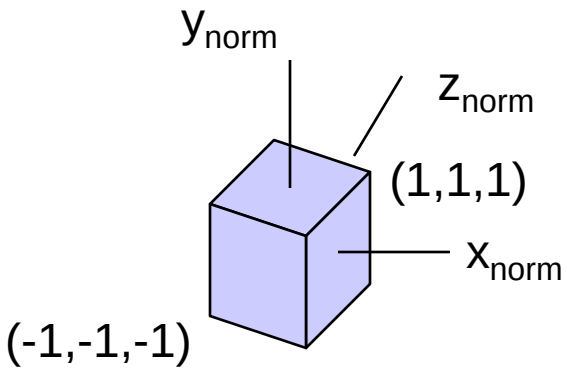
$$M_{normsymmpers} = \begin{bmatrix} \cot(\theta/2)/aspect & 0 & 0 & 0 \\ 0 & \cot(\theta/2) & 0 & 0 \\ 0 & 0 & \dfrac{z_{near}+z_{far}}{z_{near}-z_{far}} & \dfrac{-2z_{near}z_{far}}{z_{near}-z_{far}} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

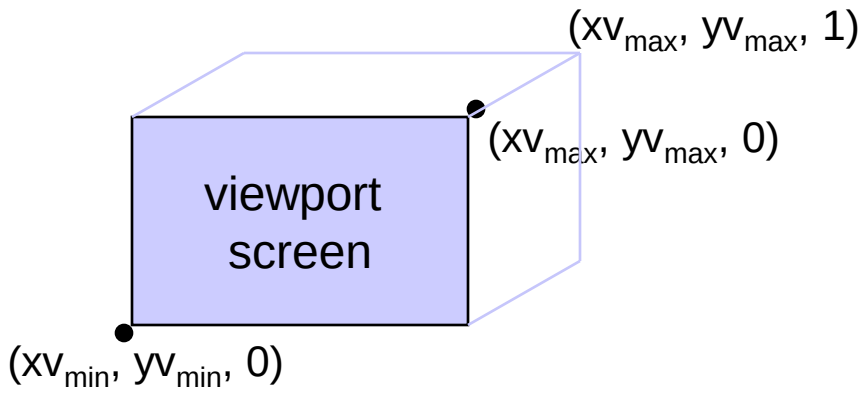
$$M_{normsymmpers} \cdot R \cdot T$$

3D Viewing Transformation Pipeline



Viewport Transformation



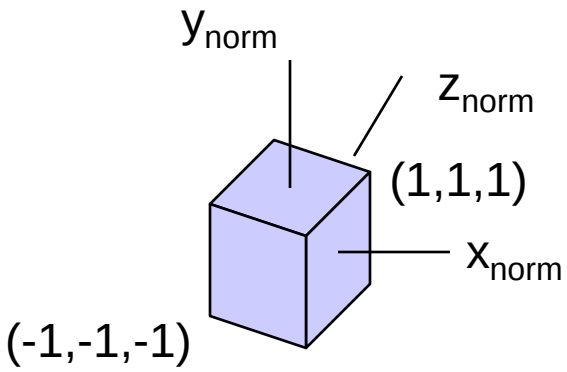


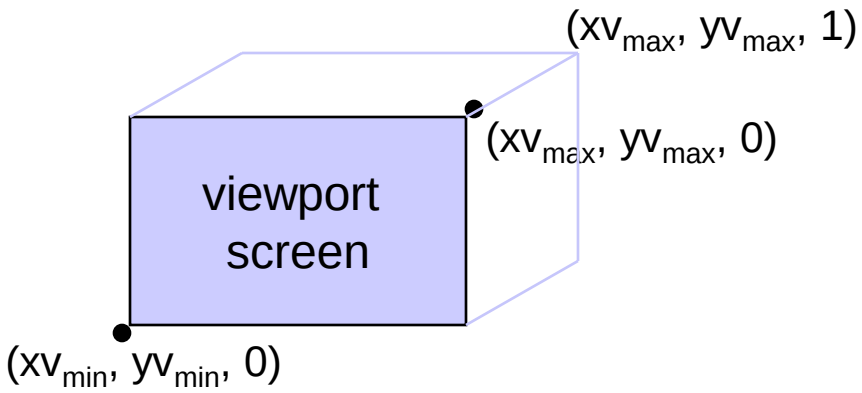
$(-1,-1,-1) \rightarrow (xv_{min}, yv_{min}, 0)$

$$\left.\begin{aligned} x_{vmin} &= S_x(-1) + t_x \\ x_{vmax} &= S_x(1) + t_x \end{aligned}\right\} \quad \begin{aligned} S_x &= \frac{xv_{max} - xv_{min}}{2} \\ t_x &= \frac{xv_{max} + xv_{min}}{2} \end{aligned}$$

$$\left.\begin{aligned} 0 &= S_z(-1) + t_z \\ 1 &= S_z(1) + t_z \end{aligned}\right\} \quad \begin{aligned} S_z &= 1/2 \\ t_z &= 1/2 \end{aligned}$$

Viewport Transformation

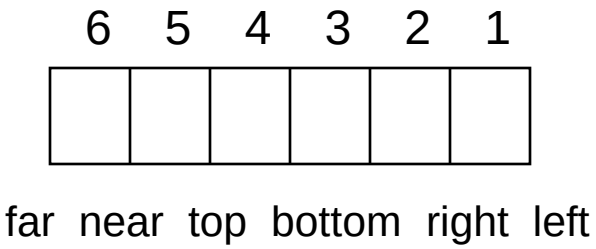


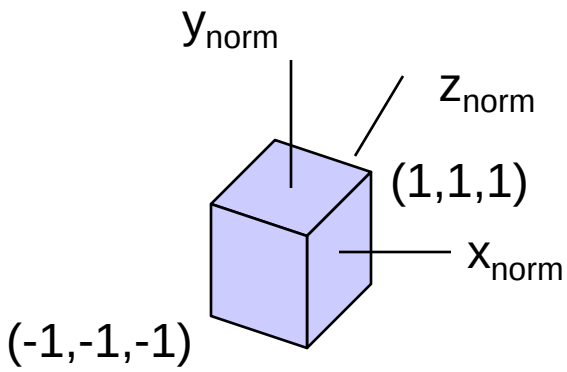


$(-1,-1,-1) \rightarrow (xv_{min}, yv_{min}, 0)$

$$M_{norm,3Dscreen} = \begin{bmatrix} \frac{x_{vmax} - x_{vmin}}{2} & 0 & 0 & \frac{x_{vmax} + x_{vmin}}{2} \\ 0 & \frac{y_{vmax} - y_{vmin}}{2} & 0 & \frac{y_{vmax} + y_{vmin}}{2} \\ 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3D Clipping





Inside if:

$-h \le x_h \le h,\ -h \le y_h \le h,\ -h \le z_h \le h$

Use sign bits of $h \pm x_h$, $h \pm y_h$, $h \pm z_h$ to set bit1-bit6

Point Clipping

Test sign bits of $h \pm x_h$, $h \pm y_h$, $h \pm z_h$

If region code is 000000 then inside

otherwise eliminate

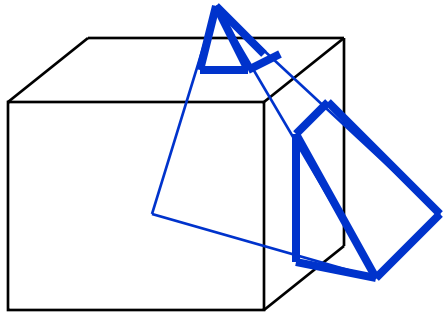
Line Clipping

1. Test region codes

- if 000000 for both endpoints => inside
- if $RC_1 \vee RC_2 = 000000$ => trivially accept
- if $RC_1 \wedge RC_2 \neq 000000$ => trivially reject

2. If a line fails the above tests, use line equation to determine whether there is an intersection

Polygon Clipping



1. Check coordinate limits of the object
   - if all limits are inside all boundaries => save the entire object
   - if all limits are outside any one of the boundaries => eliminate the entire object

2. Otherwise process the vertices of the polygons
   - Apply 2D polygon clipping
     Clip edges to obtain new vertex list.
   - Update polygon tables to add new surfaces

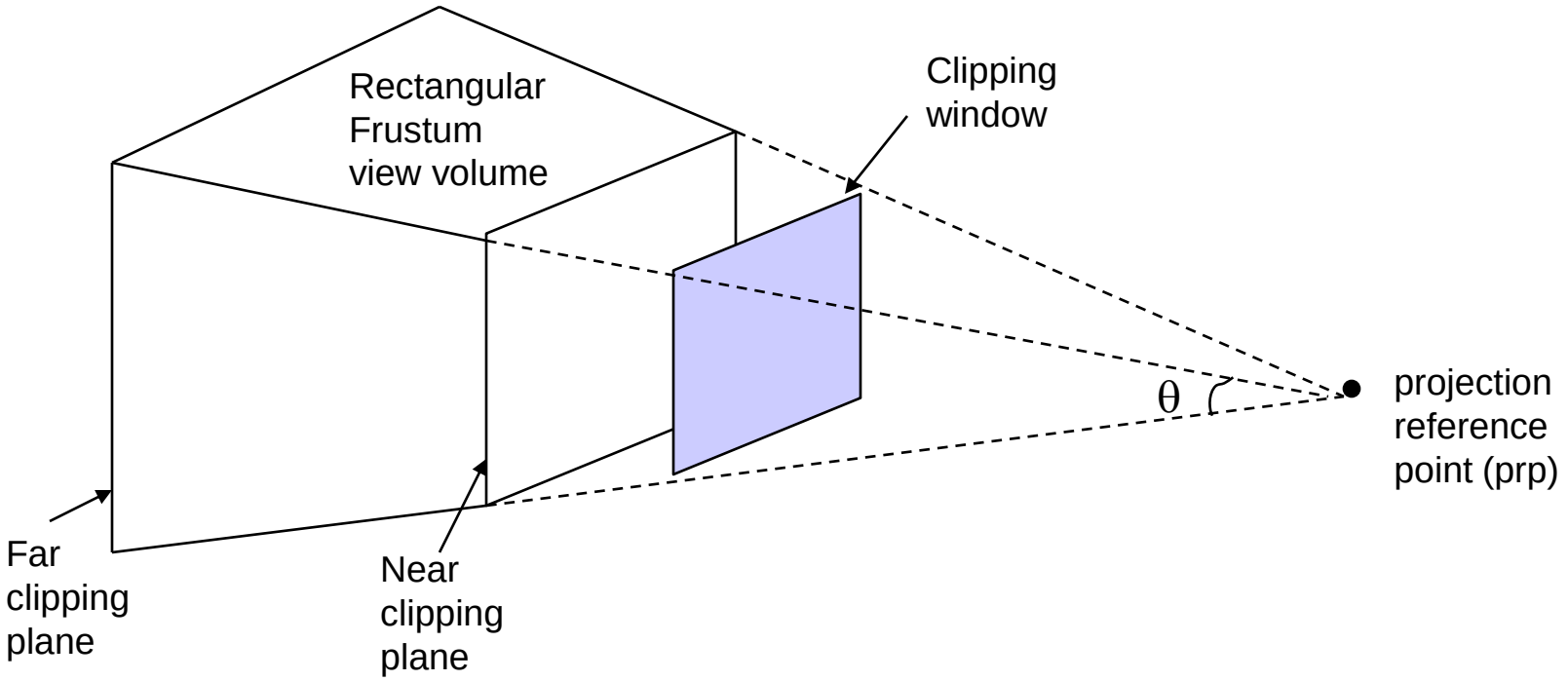
If the object consists of triangle polygons, use Sutherland-Hodgman algorithm.

OpenGL

- glMatrixMode (GL_PROJECTION)
- gluOrtho (xwmin, xwmax, ywmin, ywmax, dnear, dfar)

  Orthogonal projection
- gluPerspective (theta, aspect, dnear, dfar)
  - theta: field-of-view angle (0^0 – 180^0)
  - aspect: *aspect ratio* of the clipping window (width/height)
  - dnear, dfar: positions of the near and far planes (must have positive values)
- glFrustum (xwmin, xwmax, ywmin, ywmax, dnear, dfar)
  - if xwmin = -xwmax and ywmin = -ywmax then symmetric view volume

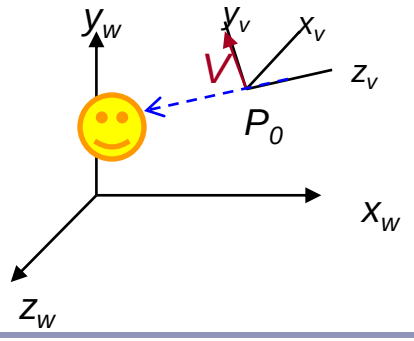
Perspective Projection



OpenGL

- glMatrixMode (GL_MODELVIEW)

- gluLookAt (x0, y0, z0, xref, yref, zref, Vx, Vy, Vz)
  Designates the origin of the viewing reference frame as,
  - the world coordinate position P0=(x0, y0, z0)
  - the reference position Pref=(xref, yref, zref) and
  - the viewup vector V=(Vx, Vy, Vz)



OpenGL

```
float eye_x, eye_y, eye_z;

void init (void) {
    glClearColor (1.0, 1.0, 1.0, 0.0);  // Set display-window color to white.

    glMatrixMode (GL_PROJECTION);       // Set projection parameters.
    glLoadIdentity();
    gluPerspective(80.0, 1.0, 50.0, 500.0);       // degree, aspect, near, far

    glMatrixMode (GL_MODELVIEW);
    eye_x = eye_y = eye_z = 60;
    gluLookAt(eye_x, eye_y, eye_z,  0,0,0,  0,1,0);    // eye, look-at, view-up
}

void main (int argc, char** argv) {
    glutInit (&argc, argv);                          // Initialize GLUT.
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);    // Set display mode.
    glutInitWindowPosition (50, 500);   // Set top-left display-window position.
    glutInitWindowSize (400, 300);      // Set display-window width and height.
    glutCreateWindow ("An Example OpenGL Program"); // Create display window.
    glViewport (0, 0, 400, 300);
    init ( );                            // Execute initialization procedure.
    glutDisplayFunc (my_objects);       // Send graphics to display window.
    glutReshapeFunc(reshape);
    glutKeyboardFunc(keybrd);
    glutMainLoop ( );                    // Display everything and wait.
}
```